Contents

Guitar Rig Analyzer — Breadboard Bring-Up Guide (v3) **2**
If something feels wrong — read this first . . . 3
How to use this guide . . . 3
What you'll have when you're done . . . 3
Master parts list — what you have on hand . . . 4
Compute / display . . . 4
Active / semiconductor . . . 4
Adapters (so SMD parts can live on the breadboard) . . . 5
Passives — through-hole, for the breadboard . . . 5
Connectors . . . 6
Breadboard / prototyping . . . 6
Tools . . . 6
Background — about your relays and the 74HC138 . . . 7

Phase 0 — Workspace and inventory **7**
Parts tray for this phase . . . 7
Steps . . . 8
End-of-phase verify . . . 8
Background . . . 9

Phase 0.5 — Solder the SMD adapters **9**
Parts tray for this phase . . . 9
Steps for the OPA1642 . . . 9
Steps for the MMBT3904 (×2) . . . 11
End-of-phase verify . . . 12
Background — why we tin a corner first . . . 12

Phase 1 — Power rails (+5V, GND, and a virtual ground) **13**
Parts tray . . . 13
Steps . . . 13
End-of-phase verify . . . 14
Background . . . 15

Phase 2 — GPIO smoke test **15**
Parts tray . . . 15
Steps . . . 16
End-of-phase verify . . . 16
Background . . . 17

Phase 3 — Jack-detect (simulated with wire jumpers) **17**
Parts tray . . . 17
Steps . . . 17
End-of-phase verify . . . 18
Background . . . 18

Phase 4 — OPA1642 buffer **19**
Parts tray . . . 19
Steps . . . 19
End-of-phase verify . . . 21
Background . . . 21

Phase 5 — First real instrument jack (Switchcraft #11) **22**
Parts tray . . . 22
Switchcraft #11 terminals . . . 22

Steps . . . 23
Solder the pigtail . . . 23
Wire into the breadboard . . . 23
Firmware invert . . . 24
End-of-phase verify . . . 24
Background . . . 24

Phase 6 — First relay driver (RELAY_SEND) 25
Parts tray . . . 25
Steps . . . 25
Firmware: temporary toggle command . . . 27
End-of-phase verify . . . 27
Background . . . 27

Phase 7 — RELAY_RETURN + signal wiring + Rig Send + Rig Return TS 28
Parts tray . . . 28
Steps . . . 28
Duplicate the driver for RELAY_RETURN . . . 28
Wire RELAY_SEND into the signal path . . . 28
Wire RELAY_RETURN into the signal path . . . 29
Add the Rig Send + Rig Return TS jacks . . . 29
End-of-phase verify . . . 29
Background . . . 30

Phase 8 — XLR return 30
Parts tray . . . 30
Steps . . . 30
End-of-phase verify . . . 32
Background . . . 33

Phase 9 — Validation harness 33
Parts tray . . . 33
Steps . . . 33
Sign-off checklist . . . 34

Appendix A — Troubleshooting decision tree 34
Power phase . . . 34
GPIO phase (smoke test, jack-detect) . . . 35
Buffer phase (Phase 4) . . . 35
Relay-driver phase (Phase 6, Phase 7) . . . 35
Jack-detect phase (Phase 5, Phase 8) . . . 36
Validation phase (Phase 9) . . . 36

Appendix B — Parts substitution table 36

Appendix C — v3 changelog 36

Guitar Rig Analyzer — Breadboard Bring-Up Guide (v3)

Document version: 3.0 (2026-06-01) — *LEGO-instruction style. Built around the parts you actually purchased. v3 fixes five critical bugs found in v2: a severed SOURCE tap and inverted relay topology in Phase 7, a stale parts-availability claim in Phase 1, a missing TQ2 pinout in Phase 6, and a draft "Wait — correction:" note in Phase 4. See the v3 changelog at the end of this document.*

v1 (breadboard_bringup.md) and **v2** (breadboard_bringup_v2.md) are kept alongside for cross-reference. v3 supersedes both.

If something feels wrong — read this first

Pin this page to your monitor.

1. **Smell something warm, see smoke, hear buzzing, or feel anything getting hot?** Unplug the USB cable from the Teensy immediately. You will not break anything by unplugging fast. You *might* break something by leaving a fault energised.
2. **Wait 30 seconds**, touch every IC with the back of your finger — if anything is uncomfortably warm, that component is suspect.
3. **Open Appendix A** (Troubleshooting tree) at the back of this doc. Most "it's broken" moments are one of five known patterns.
4. **Never trust a part until you've continuity-tested it.** A new $3 OPA1642 from DigiKey can still be DOA from a static zap during shipping.
5. **Take photos of the breadboard at the end of each phase.** When something stops working in phase 6 that worked in phase 5, the photo is the diff.

You have **5× OPA1642 spares, 100× MMBT3904 spares, 5× TQ2-5V (2 used + 3 spare), and 4× spare Chip Quik adapter boards.** Your first attempt does not have to be perfect.

> **ESD note.** Touch the silicone mat (or any grounded metal — a screw on a wall plate works) before grabbing the OPA1642 or any other IC. Don't shuffle on carpet during the SMD steps. Static you can't feel will quietly degrade a JFET input.

How to use this guide

Each phase looks like this:

1. **Goal** — one sentence.
2. **Parts tray** — a picture and a list of just the parts you need to grab from your bin for *this phase*.
3. **Steps** — numbered. Each step is *one action + one mini-diagram + one DMM check (if applicable)*.
4. **End-of-phase verify** — the test you run before moving on.
5. **Background** *(skip if you're in a hurry)* — why each piece is there, what it does, common failures.

Treat the steps like LEGO instructions: do them in order, finish each one before reading the next. The background is for when something is confusing or doesn't work — not required reading the first time through.

> **Stop sign convention.** When you see a **[STOP]** row in a verify table, **do not proceed past it** until that check passes. Breadboard work compounds — a quiet fault in phase 2 becomes an unsolvable mystery in phase 6.

What you'll have when you're done

A working **breadboard prototype** of the v0.6 signal-router hardware that will eventually live on a custom PCB. End-state block diagram:

This is the **`USE_PI_DISPLAY`** build (since you bought the Pi 5 + touch display). The Teensy still owns all DSP and routing; the Pi just renders.

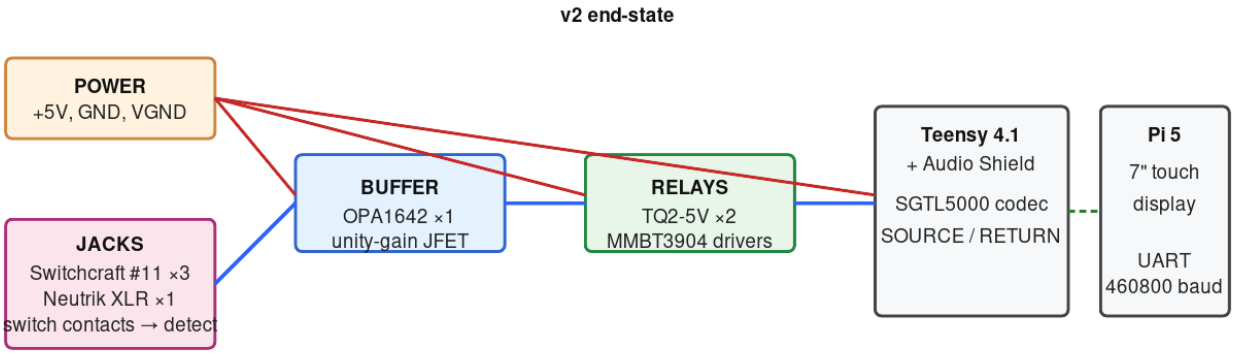


Figure 1: v2 → v3 end-state block diagram

Master parts list — what you have on hand

This consolidates your **DigiKey + Mouser + Vilros + Amazon** orders into one place. I've marked which phase each part is used in, so you can lay out a bin per phase.

Compute / display

Part	Source	Used in
PJRC Teensy 4.1 (no pins) ×1	Amazon	Phase 0 onward
Teensy 4 Audio Shield Rev D ×1	Amazon	Phase 4 onward
SanDisk 128GB microSD ×1	Amazon	Optional (calibration / exports)
Raspberry Pi 5 8GB ×1	Vilros	Phase 9 + setup
Pi 5 Active Cooler ×1	Vilros	Phase 9
Pi 27W USB-C PSU ×1	Vilros	Phase 9
Pi Touch Display 2 (7") ×1	Vilros	Phase 9
Freenove RPi GPIO breakout HAT ×1	Amazon	Phase 9 (Pi↔Teensy UART)
HiLetgo SSD1309 2.42" OLED ×1	Amazon	Optional secondary status display

Active / semiconductor

Part	Qty	Source	Used in
OPA1642AID (SOIC-8)	5	DigiKey 296-25614-5-ND	Phase 4 (buffer)
MMBT3904 (SOT-23 NPN)	100	DigiKey	Phase 6, 7 (relay drivers)
TQ2-5V (non-latching DPDT, 5V) — to buy	2	DK 255-1004-5-ND	Phase 6, 7 (relays)
TQ2-L2-5V (latching DPDT — keep for future PCB)	10	DigiKey	*Not used in breadboard* — see "About your relays" sidebar

Part	Qty	Source	Used in
1N4148 DO-35 (through-hole)	20	Amazon	Phase 6, 7 (flyback)
74HC138 (SO-16 decoder)	25	DigiKey	*Not used in breadboard* — see "About the 74HC138" sidebar

Adapters (so SMD parts can live on the breadboard)

Part	Qty	Source	Used in
Chip Quik PA0001 SOIC-8 to DIP-8	2	Mouser 910-PA0001	Phase 0.5 (solder OPA1642 onto this)
Chip Quik SIPH06-1 SOT-23 to DIP	4	Mouser 910-SIPH06-1	Phase 0.5 (solder MMBT3904 onto this)

Passives — through-hole, for the breadboard

Part	Qty	Source	Used in
10µF / 25V radial electrolytic (Panasonic ECA-1EM100)	10	Mouser	Phase 1, 4 (decoupling, VGND, AC coupling)
0.1µF / 50V leaded ceramic (KYOCERA AVX SR205C104KART)	20	Mouser	Phase 1, 4 (decoupling)
10nF / 50V leaded ceramic (Vishay K103K10X7RF5UL2)	10	Mouser	Phase 4 (input AC coupling)
1nF / 50V leaded C0G (Vishay K102J15C0GF53H5G)	20	Mouser	Optional — RC filter calibration in Phase 9
Through-hole resistors (YAGEO MFR-25 series)	various	Mouser	Phases 2–7
└ 47Ω 1%	40		(HF damping if needed)
└ 100Ω 1%	40	Phase 2 (LED current limit)	
└ 1kΩ 1%	60	Phase 4 (input divider option)	
└ 10kΩ 1%	60	Phase 1 (VGND divider), Phase 6/7 (BJT pulldown)	
└ 47kΩ 1%	40	Reserve	

Part	Qty	Source	Used in
└ 100kΩ 1%	50	Phase 4 (input bias, substitute for 1MΩ)	

Connectors

Part	Qty	Source	Used in
Switchcraft #11 1/4” mono jack	10	Amazon	Phase 5, 7 (instrument in, rig send, rig return TS)
Neutrik NC3FAH2-0 XLR F/M combo	2	Mouser	Phase 8 (XLR return)
Phoenix Contact 1715721 2P 5.08mm terminal block	4	Mouser + DK	Reserve (PCB output)
JST B2B-XH-A 2P 2.5mm header	10	DigiKey	Reserve (PCB output)

Breadboard / prototyping

Part	Source	Used in
BB830 solderless breadboard	Amazon	Phase 0 onward
560pc jumper-wire kit	Amazon	Phase 0 onward
WWZMDiB 2.54mm perf board ×54	Amazon	Reserve (post-breadboard hand-built version)
ElectroCookie prototype boards ×6	Amazon	Reserve
Terminal-block breakout for Teensy 4.1	Amazon	Phase 0 (screw-down access to Teensy pins)
Custom Circuit Solutions 16-pin SMD breakouts ×6	Amazon	Reserve (for 74HC138 on the PCB)
Cermant SMD 0402/0603/0805 breakout plates ×50	Amazon	Reserve (PCB-stage if needed)

Tools

Tool	Source
FNIRSI DWS-200 200W soldering station (with F210 + F245 tips)	Amazon
Kester 63/37 0.020” no-clean solder	Amazon
ENGINEER SS-03 desolder pump	Amazon
iFixit precision tweezers	Amazon
Klein MM420 auto-ranging TRMS DMM	Amazon
KOTTO solder fume extractor	Amazon

Tool	Source
VECO-T silicone soldering mat (heat-resistant)	Amazon
99.9% IPA, 1 gal	Amazon
DIDODI SMT 4-in-1 soldering practice kit	Amazon

Background — about your relays and the 74HC138

> *Skip this section the first time through. It's here so you know why some parts you ordered aren't in the bring-up path.*

About your relays. You ordered **TQ2-L2-5V** ×10. The `-L2` suffix means **dual-coil latching DPDT**. They take a *pulse* on a "set" coil to switch one way, and a pulse on a "reset" coil to switch the other — then they *hold state with zero current*. That's great for production: quieter, lower power, survives brief power glitches. But the firmware in `signal_router.cpp` today drives the relay statically (`digitalWrite(PIN, HIGH/LOW)`), so a latching relay would need both new firmware *and* a second GPIO per relay. For the breadboard, buy **2× TQ2-5V** (non-latching, same family and footprint — DigiKey 255-1004-5-ND). Keep the TQ2-L2 for a future PCB revision; we can revisit the firmware then.

About the 74HC138. You ordered 25 of them. The 74HC138 is a **3-to-8 line decoder/demux**. It's not used in the breadboard bring-up — it's there for the PCB phase if you decide to multiplex relay control or jack-detect signals across fewer Teensy pins (3 select lines → 8 outputs). For 2 relays + 4 jack-detect inputs, you don't need it; you have plenty of free GPIOs. Save them.

About the OLED. The HiLetgo SSD1309 is a 128×64 monochrome OLED. Not needed for the bring-up. It would make a nice secondary "status badge" display if you ever want one on a hardware front-panel separate from the Pi touchscreen.

Phase 0 — Workspace and inventory

> **Goal:** A clean, well-lit workspace with the Teensy on standby and every part for phases 1–9 sorted into labeled cups or bins.

Parts tray for this phase

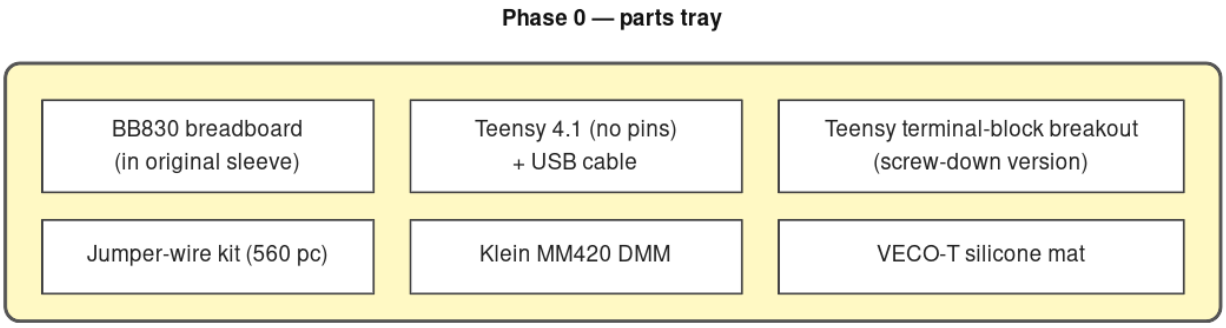


Figure 2: Phase 0 — parts tray

Steps

1. Lay out the silicone mat on a flat surface. This is your work area for the next 9 phases. Keep food, drinks, and pets away.

2. Sort your parts into cups. Use the master parts list above as your guide. Label each cup with the part name + the phase number. Suggested groupings:

- **Cup A — phase 1 power:** 10µF caps (×3), 0.1µF caps (×4), 10K resistors (×2).
- **Cup B — phase 2 LEDs:** any 2 LEDs you have lying around (an old breadboard kit, an old toy), 100Ω resistors (×2).
- **Cup C — phase 4 buffer:** 1× OPA1642AID, 1× Chip Quik PA0001 adapter (still bare), 100K resistors (×2), 10nF ceramic (×2), 10µF electrolytic (×2), 0.1µF ceramic (×2).
- **Cup D — phase 6/7 drivers:** 4× MMBT3904, 4× SIPH06-1 adapters (still bare), 1N4148s (×4), 10K resistors (×4), 100Ω resistors (×4 — base limiters; we'll explain).
- **Cup E — phase 5/7/8 jacks:** 3× Switchcraft #11, 1× Neutrik XLR.

3. Mount the Teensy on the screw-down breakout. Push the Teensy 4.1 (no pins) onto the breakout board's pin sockets. The Teensy is *not* on the breadboard — the breakout sits next to it.

4. Power up the Teensy on USB and confirm it boots. Open the Arduino IDE Serial Monitor at 115200 baud. You should see `[Main] Guitar Rig Analyzer v0.5.0 ...`. If not, fix that first (see README.md).

5. Bridge top + bottom power rails on the BB830 with two short jumpers each side. Cut 4 short pieces of solid wire — 2 red, 2 black. Bridge the top-red rail to the bottom-red rail (one bridge at column 1, one at column 60). Same for top-blue ↔ bottom-blue.

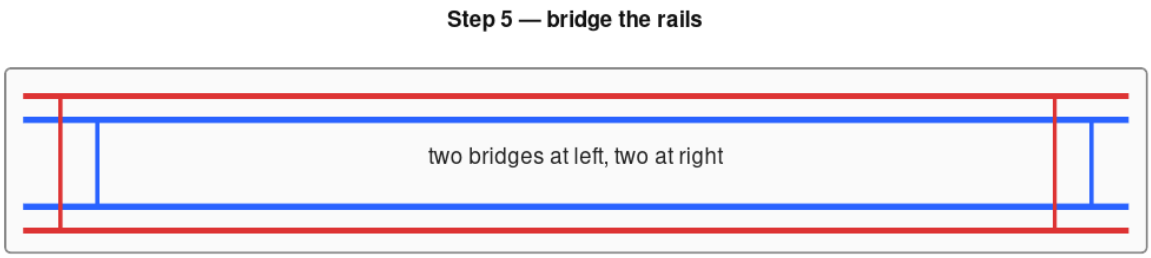


Figure 3: Phase 0 step 5 — bridge the rails

End-of-phase verify

[STOP] Check	How	Pass
Top-red ↔ bottom-red bridged	DMM continuity, top-red anywhere to bottom-red anywhere	Beep
Top-blue ↔ bottom-blue bridged	Same, on the blue rails	Beep
Red and blue rails are NOT shorted to each other	DMM continuity, top-red to top-blue	**No beep**
Teensy boots normally	Serial Monitor, 115200 baud	`[Main] Guitar Rig Analyzer v0.5.0 ...` banner

Background

The BB830 has *four* independent power rails (two top, two bottom). Each runs the full length of the board. They are **not** connected to each other from the factory — the manufacturer leaves that to you so you can use them as four independent rails if you want. We only need two rails (one +5V and one GND, plus a third VGND coming in Phase 1), so we bridge the top rails to the bottom rails for redundancy. Two bridges per side keeps the rails at the same potential even with the inductance and resistance of long jumper-wire runs.

Phase 0.5 — Solder the SMD adapters

> **Goal:** Turn your SOIC-8 OPA1642 and your SOT-23 MMBT3904s into "DIP-like" parts you can push into the breadboard. You'll solder them onto Chip Quik adapter boards. This is the only phase that requires a soldering iron — every later phase is purely plug-in.
>
> **First time soldering SMD?** Spend an hour on the **DIDODI SMT practice kit** before touching the OPA1642. Your $3 chip will thank you. Practice on the 0805 resistors and SOIC-8 footprints from the kit until you can do them cleanly without bridging pins.

Parts tray for this phase

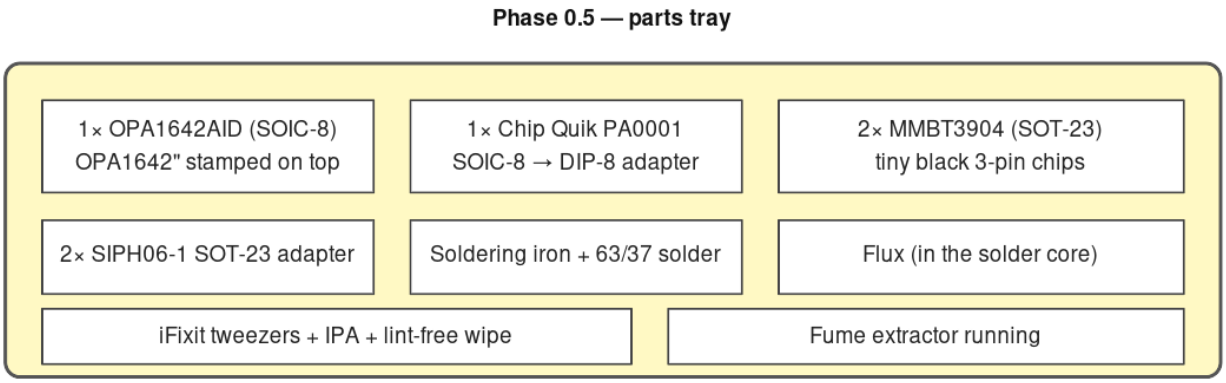


Figure 4: Phase 0.5 — parts tray

Steps for the OPA1642

1. Identify pin 1 on the OPA1642. Look closely at the top of the chip. You'll see a small **circular dimple** or **dot of laser-etched ink** in one corner. That corner is **pin 1**. The pins count anti-clockwise from there when viewed from above.

2. Look at your PA0001 adapter board. It has a SOIC-8 footprint on top and a row of DIP-8 holes around the edge. The pin-1 corner of the SOIC footprint is silkscreened with a "1" or a square pad (the other pads are oval).

3. Pre-tin one corner pad of the SOIC footprint. Heat the pad, touch solder, lift. You want a small bead — just enough to hold the chip in place.

4. Place the OPA1642 on the footprint. Use tweezers. Make sure pin 1 (the dot corner) lines up with pin 1 of the footprint. Press lightly while you reflow the tinned pad to tack it down.

5. Solder the diagonally-opposite pin. Now the chip is held at both corners and can't shift.

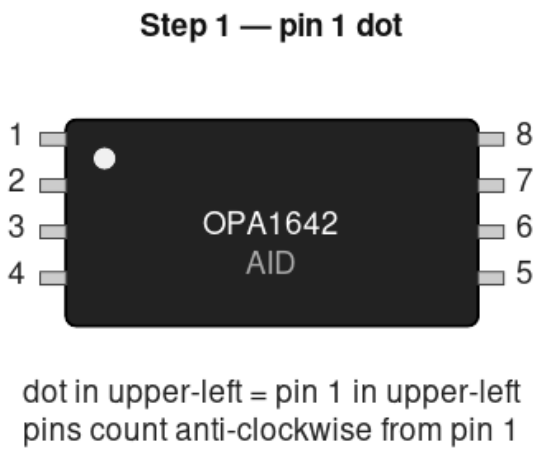


Figure 5: OPA1642 — pin 1 dot

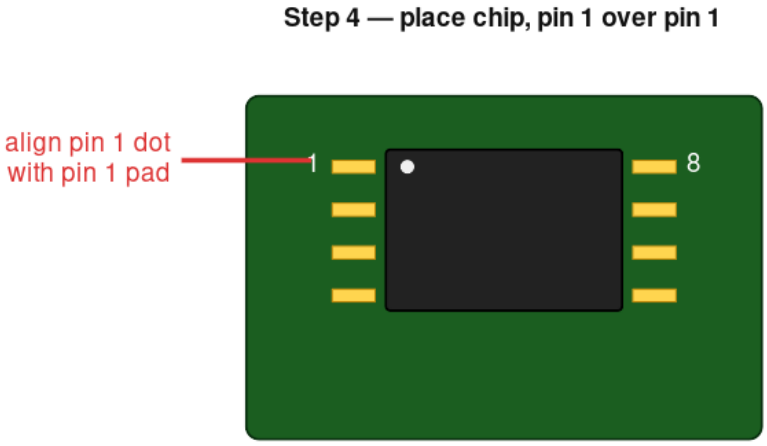


Figure 6: OPA1642 placed on PA0001 adapter

6. Solder the remaining 6 pins. Touch the iron tip to the *side* of the pad-and-pin joint (not the top — that just transfers heat to the chip body). Touch solder to the opposite side. It'll flow in. Move to the next pin.

7. Inspect under good light. Each joint should be a small, shiny *fillet* — a triangle of solder bridging pad to pin. Dull or "grainy" joints mean it didn't get hot enough. Big blobs mean too much solder.

8. Look for bridges between pins. Pins are 1.27mm apart on SOIC-8. A bridge looks like a solder *valley* connecting two pins. To fix: re-flow with extra flux, then wick with desoldering braid (or use the SS-03 pump). Do not leave bridges — they short the chip's pins together and can destroy it on power-up.

9. Clean the flux residue. Dampen a lint-free wipe with IPA and gently scrub the adapter. The residue should lift off as a brown haze.

10. Solder the DIP-8 header pins into the outside row of holes. Most PA0001 boards come with the header pins separate. Push them through from the *opposite* side of the board (so the long pins stick out where they'll plug into the breadboard, and the short pins solder on the side with the SOIC chip).

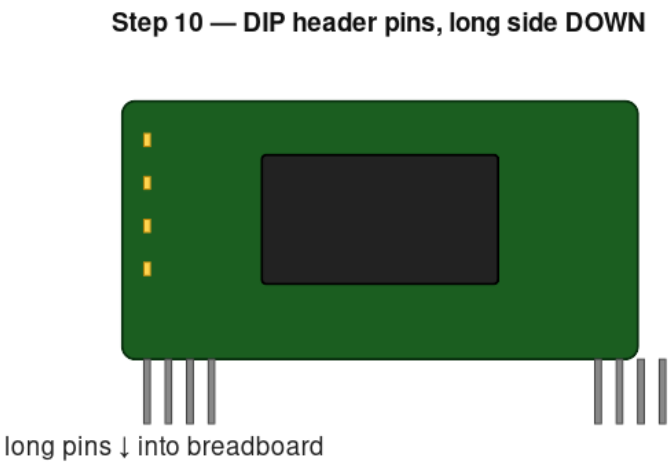


Figure 7: SIPH06-1 DIP header pins, long side DOWN

Steps for the MMBT3904 (×2)

The MMBT3904 is a SOT-23 part — even smaller, with 3 pins. Same technique, more practice.

11. Identify the pinout. The MMBT3904 SOT-23 footprint has the *single pin* on one side and *two pins* on the other. Looking at it with the markings up and the single pin to the right:

- Bottom-left pin = **Base (B)**
- Top-left pin = **Collector (C)**
- Right pin = **Emitter (E)**

12. Look at your SIPH06-1 adapter. It has a SOT-23 footprint on one face and a short row of DIP-style pin holes. Note the C / B / E silkscreen markings.

13. Tin one of the SOT-23 pads, place the part, tack it down, then solder the remaining pins. Same technique as the OPA1642 — just smaller. Steady hands.

14. Solder DIP header pins into the row of holes. Same as step 10.

15. Test continuity from each header pin to the corresponding C/B/E label. DMM in continuity mode: - Header pin labeled C ↔ SOT-23 pin C → **beep** - Header pin labeled B ↔ SOT-

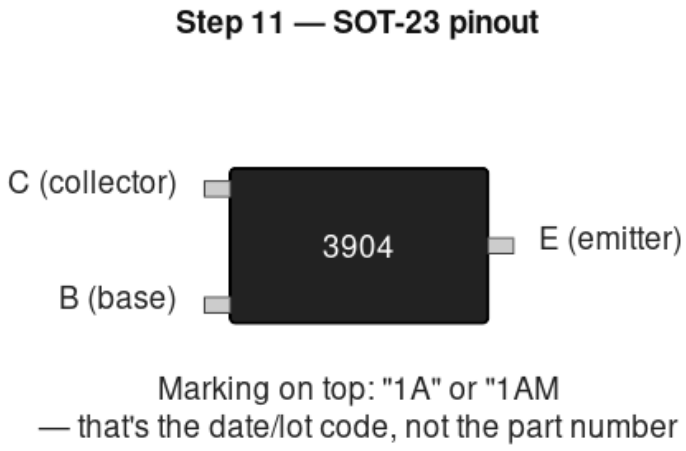


Figure 8: MMBT3904 SOT-23 pinout

23 pin B → **beep** - Header pin labeled E ↔ SOT-23 pin E → **beep** - Any two header pins ↔ each other → **no beep** (no shorts)

16. Repeat for the second MMBT3904. You need 2 for phases 6 + 7. Solder 2 of the 4 SIPH06-1 adapters you bought; keep 2 as spares.

End-of-phase verify

[STOP] Check	How	Pass
OPA1642 adapter — pin 1 to header pin 1	DMM continuity	Beep
OPA1642 adapter — no pin-to-pin shorts (any adjacent pair)	DMM continuity, 1↔2, 2↔3, 3↔4, 5↔6, 6↔7, 7↔8	**No beep** on any pair
MMBT3904 adapter #1 — each of C/B/E reads through	DMM continuity to header pin	Beep
MMBT3904 adapter #1 — no C↔B, C↔E, B↔E shorts	DMM continuity on each pair	**No beep**
MMBT3904 BJT test (sanity)	DMM in diode mode: red on B, black on C → 0.65–0.75V drop. Red on B, black on E → 0.65–0.75V drop. Any other direction → OL.	All four readings match
MMBT3904 adapter #2 — same		

Background — why we tin a corner first

Surface-mount soldering relies on a technique called “tack and stitch”: you create one small dab of solder on a corner pad, then re-melt it while pressing the chip in place. That single pin now anchors the chip — both physically and electrically. Once anchored, you solder the diagonally-opposite pin (so the chip can’t lift on one side), and only then do you solder the remaining pins. If you try to solder all 8 pins in order from one side, the chip will heat unevenly and “tombstone” — one end lifting off the pad.

Heat too long on one pin? SOIC-8 plastic packages can take ~260°C for ~10 seconds without damage. If you're on a pin for more than 5 seconds and the solder still won't flow, the iron temperature is too low (set the FNIRSI to 350°C for leaded solder) or the pad is contaminated (clean with flux + IPA).

SOT-23 vs SOIC-8 difficulty. SOT-23 is smaller but has only 3 pins. SOIC-8 has 8 pins on 1.27mm pitch. Most people find SOT-23 *easier* on their first try because each pin is more accessible.

Phase 1 — Power rails (+5V, GND, and a virtual ground)

> **Goal:** Distribute +5V from the Teensy USB to the breadboard, generate a 2.5V virtual ground (VGND) for the op-amp in phase 4, and decouple everything.
>
> **What changed from v1 (corrected in v3).** You have **5× TLE2426CLP rail-splitter ICs on hand** (Mouser 595-TLE2426CLP, see `parts_inventory.md`). We're *not* using them on the breadboard — they're being saved for the production PCB. The breadboard uses a **two-resistor divider** (10K + 10K) with a 10µF reservoir cap instead, for two reasons: (1) it lets you probe each side of the divider independently with the DMM, which is a more educational debug experience; (2) it preserves all 5 TLE2426s for the final board run, where their lower noise actually matters. If you'd rather use the TLE2426 now, see the substitution note in Appendix B.

Parts tray

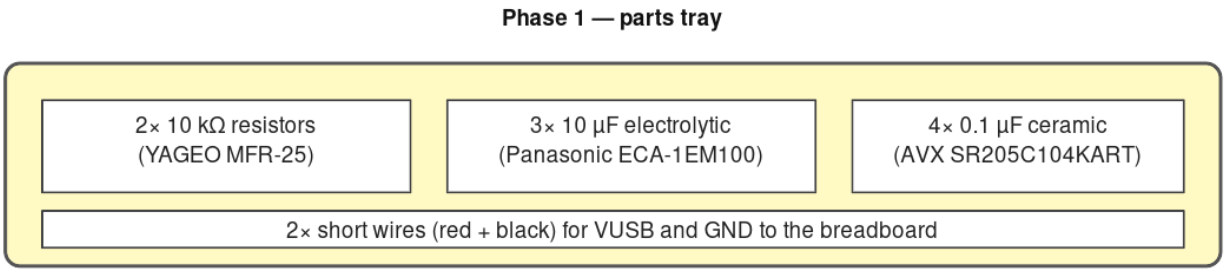


Figure 9: Phase 1 — parts tray

Steps

1. Unplug the Teensy from USB. Everything in phase 1 happens cold.

2. Run a red wire from Teensy `VUSB` pad → top-red rail. The VUSB pad on the Teensy 4.1 is at the very edge of the board, near the USB-C connector. Solder a short pin if the breakout doesn't expose VUSB, or use the screw terminal on your Teensy breakout. Tuck the other end of the wire into the top-red rail anywhere near column 1.

3. Run a black wire from any Teensy `GND` → top-blue rail. Any GND pin works. Top-blue rail anywhere near column 1.

4. Place the first decoupling cap on +5V near column 5. A 10µF electrolytic. The longer leg (the **+** side) goes into the top-red rail; the shorter leg (the **−** side, also marked with a stripe down that side of the cylinder) goes into the top-blue rail.

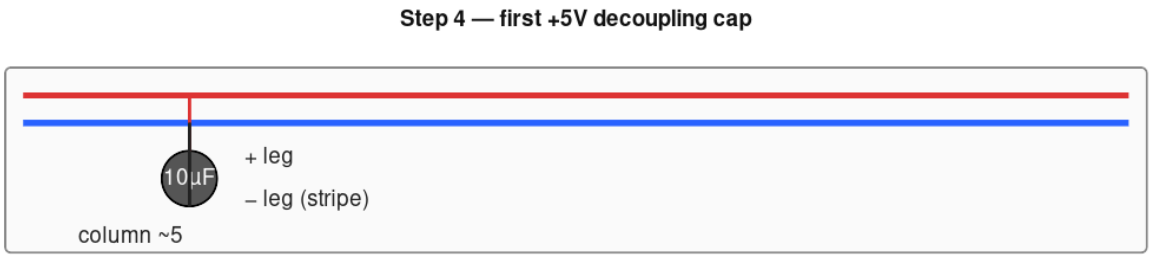


Figure 10: Phase 1 step 4 — first +5V decoupling cap

5. Place a 0.1 µF ceramic cap right next to the electrolytic. Ceramic caps are not polarized — orientation doesn't matter. Put it at column 7, one leg in top-red, the other in top-blue.

6. Build the VGND divider in column 15. - Push one leg of a 10 kΩ resistor into the top-red rail; push the other leg into hole 15a. - Push one leg of a second 10 kΩ resistor into hole 15b; push its other leg into the top-blue rail.

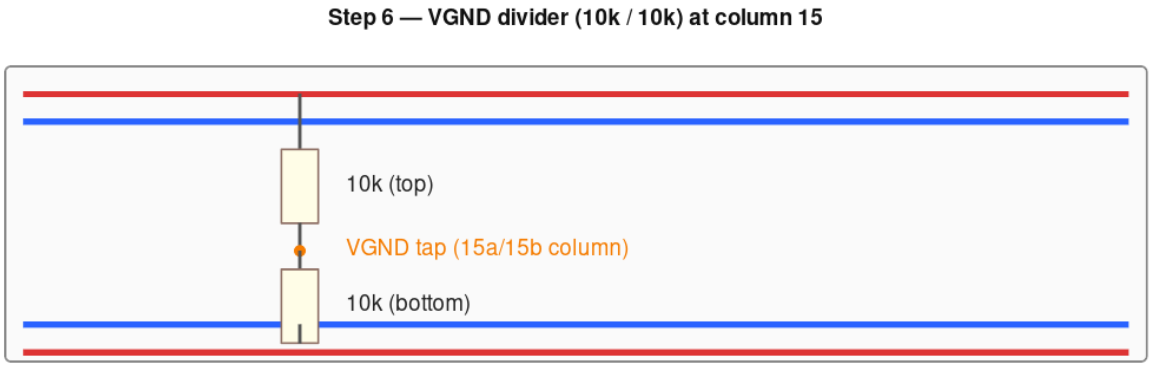


Figure 11: Phase 1 step 6 — VGND divider (10K / 10K)

7. Run a wire from the VGND tap (column 15) to the bottom-red rail. This *re-purposes the bottom-red rail as VGND.* It's painted red but it's not +5V anymore. **Stick a piece of masking tape on the breadboard right next to the bottom-red rail and write "VGND ~2.5V" on it.**

8. Place a 10 µF cap from VGND to GND at column 18. Longer leg (+) into the bottom-red (VGND), shorter leg (−) into the bottom-blue (GND). This is your VGND reservoir — it stabilizes the divider against transient loads from the op-amp.

9. Place a 0.1 µF ceramic at column 20, also VGND to GND. For high-frequency decoupling.

End-of-phase verify

Plug in USB. **Nothing should hum, smoke, or get warm.** Then DMM:

[STOP] Check	DMM probes	Pass
+5V rail is alive	red → top-red rail, black → top-blue (GND)	4.85 – 5.10 V DC

[STOP] Check	DMM probes	Pass
VGND is ≈ 2.5 V	red → bottom-red rail (the VGND rail), black → top-blue (GND)	2.40 – 2.60 V DC
VGND really is midway	red → top-red, black → VGND. Then red → VGND, black → GND. Both should be within 0.05 V of each other.	Yes
GND continuity	continuity, top-blue ↔ bottom-blue at multiple spots	Beep everywhere
No power-to-GND shorts	continuity, top-red ↔ top-blue	**No beep**

Background

A **virtual ground** is just a stable voltage at the midpoint of your single supply that the analog circuit treats as "zero." The op-amp's inputs and outputs swing symmetrically above and below it — so on a +5V/GND system, an audio signal that wants to swing ±1V will sit between 1.5V and 3.5V (centered on the 2.5V VGND), never touching either rail.

The 10K/10K divider works but has one weakness: its output impedance is 5K (= 10K || 10K). Any current the op-amp pulls or pushes from its bias resistor *to* VGND moves the VGND voltage. We hide this with the 10µF reservoir cap, which "remembers" the right voltage and supplies (or absorbs) transient current from the divider node. For breadboard work this is fine. The final PCB will use a proper rail-splitter IC (TLE2426 or similar) for clean noise performance.

About the missing 1MΩ. In phase 4 you'll use 100K as the OPA1642 input-bias resistor instead of 1M (the value v1 of this doc called for). The implication: input impedance to the buffer is ~100K instead of 1M. For most guitar pickups this is *fine* — measurable but minimal high-frequency rolloff. For very high-impedance vintage humbuckers you might notice ~0.5 dB cut at 10kHz. Easy to fix later by chaining 10× 100K resistors in series. Don't worry about it for now.

Phase 2 — GPIO smoke test

> **Goal:** Prove pins 14 and 15 (the eventual relay control pins) can be driven HIGH/LOW from firmware. Use any 2 LEDs you have on hand.

Parts tray

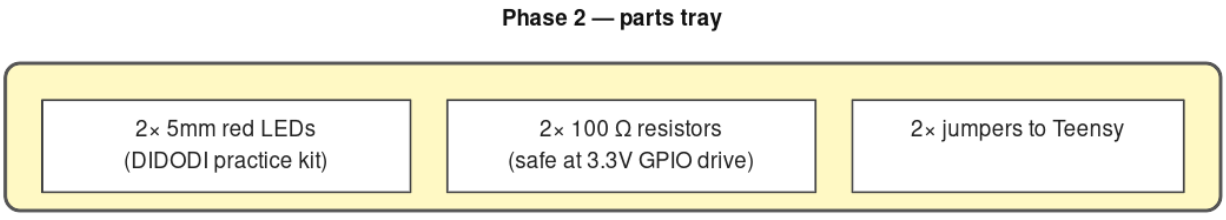


Figure 12: Phase 2 — parts tray

Steps

1. **Unplug USB.**
2. **Place LED #1 at column 30.** Long leg (anode, +) in `30a`, short leg (cathode, −) in `33a`.
3. **Jumper LED #1 cathode to GND.** Wire from `33a` to top-blue rail.
4. **Place a 100Ω resistor between LED #1 anode and column 27.** One leg in `30a` (shared with the LED anode through the column strip), the other leg in `27a`.
5. **Jumper from `27a` → Teensy pin 14.**

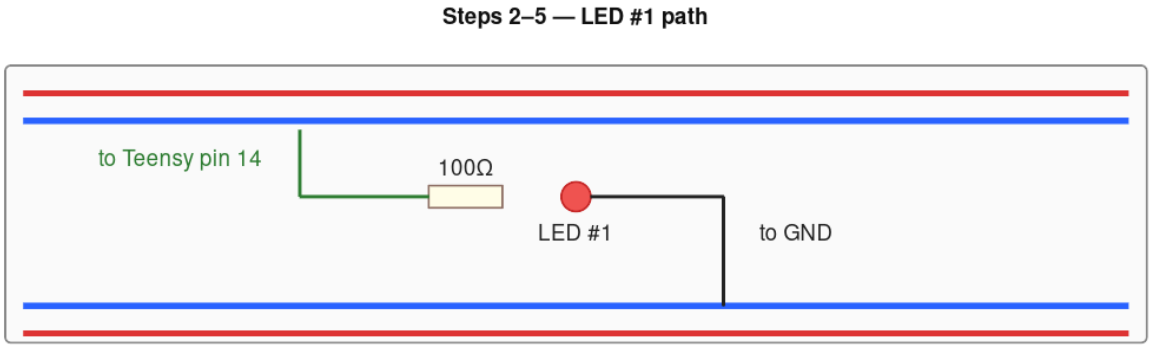


Figure 13: Phase 2 — LED #1 wiring path

6. **Repeat steps 2–5 for LED #2 at column 40, wired to Teensy pin 15.**
7. **Add the smoke-test code block to `setup()`.** Open `guitar_rig_analyzer.ino`. Find the end of `setup()` (right before `loop()` starts). Add:

```
// ==== PHASE 2 SMOKE TEST — REMOVE BEFORE PHASE 3 =====
Serial.println("[BringUp] Phase 2 LED smoke test — 5 sec");
pinMode(RELAY_SEND_PIN,   OUTPUT);
pinMode(RELAY_RETURN_PIN, OUTPUT);
for (int i = 0; i < 5; ++i) {
    digitalWrite(RELAY_SEND_PIN,   HIGH);
    digitalWrite(RELAY_RETURN_PIN, LOW);
    delay(500);
    digitalWrite(RELAY_SEND_PIN,   LOW);
    digitalWrite(RELAY_RETURN_PIN, HIGH);
    delay(500);
}
digitalWrite(RELAY_SEND_PIN,   LOW);
digitalWrite(RELAY_RETURN_PIN, LOW);
Serial.println("[BringUp] Phase 2 smoke test done");
// ====================================================
```

8. **Build and flash.**

```
arduino-cli compile --fqbn teensy:avr:teensy41 .
arduino-cli upload  --fqbn teensy:avr:teensy41 .
```

9. **Open the Serial monitor at 115200 baud and watch the LEDs.**

End-of-phase verify

[STOP] Check	Expected
Both LEDs alternate at 1 Hz for 5 seconds after every boot	Yes
Serial monitor prints the bring-up banner lines	`[BringUp] Phase 2 ...` lines appear
After 5 seconds both LEDs go OFF	Yes (pins return to LOW)
DMM on pin 14 to GND during the blink	Alternates 0 V ↔ 3.3 V

10. Remove the smoke-test block from `setup()`. Leave the LEDs and resistors on the breadboard — they become diagnostic indicators in phase 6 (in parallel with the relay coils).

Background

Why 100Ω? Teensy 4.1 GPIOs drive ~3.3V. A red LED has a forward voltage of about 2V and a typical operating current of 5–20 mA. Resistor value to set 15 mA: (3.3 − 2.0) / 0.015 = 87Ω. We round up to 100Ω (which gives about 13 mA — perfectly visible, well within the GPIO's ~10 mA continuous rating per pin). The v1 doc called for 330Ω, which is sized for 5V GPIO and gives a slightly dim 4 mA on a 3.3V system — fine but barely visible in a bright room. 100Ω is brighter and still safe.

Why drive the LED directly from the GPIO? It's a smoke test for the GPIO itself. In phase 6 we'll move the LED in parallel with the relay coil and let the transistor handle the LED's current too — but for now, GPIO → resistor → LED is the simplest possible end-to-end test.

Phase 3 — Jack-detect (simulated with wire jumpers)

> **Goal:** Confirm the firmware sees four GPIO inputs go LOW when momentarily grounded. No DIP switch needed — a short wire jumper to GND, touched briefly, is enough.

Parts tray

Just **one black wire**. That's the whole tray.

Steps

1. Make sure USB is plugged in and the firmware is running. No code changes for this phase — `signalRouter_init()` already configures these pins as `INPUT_PULLUP`.

2. Strip 5 mm off both ends of a 15 cm black jumper wire. Push one end firmly into the top-blue (GND) rail.

3. Open the Serial monitor at 115200 baud. You'll watch for `JACK_CHANGED` events as you tap the free end of the wire against each Teensy pin.

4. Briefly touch the free end of the jumper to Teensy pin 16 (JACK_INSTRUMENT_PIN). Hold for ~200 ms, then release.

Expected output:

```
[Main] signalRouter: JACK_CHANGED  (no return, no injection)
```

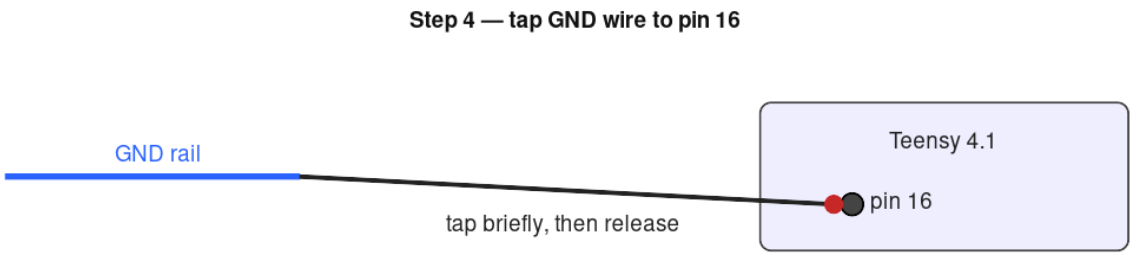


Figure 14: Phase 3 — tap GND wire to Teensy pin 16

5. Repeat for pins 17, 22, and 24. One tap per pin. Wait ~1 second between taps to keep the Serial output readable.

Touch this pin	Expected serial event
16 (Instrument)	`JACK_CHANGED (no return, no injection)`
17 (Rig Send)	`JACK_CHANGED (no return, no injection)`
22 (Return TS)	`JACK_CHANGED (TS return, no injection) + ROUTE_CHANGED`
24 (Return XLR)	`JACK_CHANGED (XLR return, no injection) + ROUTE_CHANGED`

6. DMM sanity check on one pin while it's NOT being grounded. Red probe on Teensy pin 16, black on GND. Reading: ~3.3V (the internal pull-up).

End-of-phase verify

[STOP] Check	Expected
Each of pins 16, 17, 22, 24 generates a `JACK_CHANGED` event when momentarily tapped to GND	Yes, all four
Pin 22 (TS) or pin 24 (XLR) also generates a `ROUTE_CHANGED` event	Yes
DMM on any of these pins with no tap shows ~3.3V (INPUT_PULLUP active)	Yes
With pin tapped to GND, DMM shows < 0.1V	Yes

Background

Why active-low (LOW = jack inserted)? Because the Switchcraft #11 jacks you bought (and most guitar jacks) use a switch contact that closes to ground when the jack is *empty*, opening when a plug is inserted. With `INPUT_PULLUP`, an empty jack would pull the pin LOW (closed-to-tip-to-sleeve-to-GND), and an inserted plug would let the pin float HIGH (pulled up by the internal 50K resistor).

Wait — that's backwards from what we just tested. Right. The firmware's *current* convention is "LOW = inserted." That works directly with jacks that have a normally-open switch (the Cliff CL1384 style). For Switchcraft #11s, we'll need a one-line firmware invert in phase 5. We're testing the GPIO + debounce machinery here; we'll fix the sense convention when we wire the real jacks.

Phase 4 — OPA1642 buffer

Goal: Build a unity-gain JFET buffer that takes a high-impedance signal in and presents a low-impedance copy out, AC-coupled at both ends. Drive it with a phone headphone jack as a test source; confirm the live spectrum on the Audio Shield lights up.

What changed from v1. Using 100K input bias (you don't have 1M); using 10nF input coupling (you have these; 1µF was the v1 spec but works similarly with 100K bias — gives a 160 Hz corner; for guitar use we want lower, see "Background"). VGND comes from the resistor divider you built in phase 1.

Parts tray

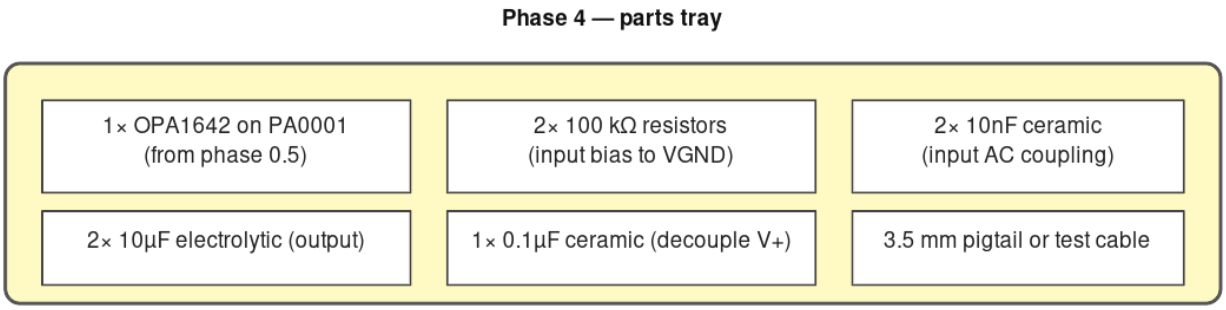


Figure 15: Phase 4 — parts tray

Steps

1. Unplug USB.

2. Place your OPA1642 (on its PA0001 adapter) at columns 35–38, straddling the center gap. Notch / pin-1 marker on the **left side**. The DIP-8 layout means:

```
        notch
1 ──┤     ├── 8
2 ──┤     ├── 7
3 ──┤     ├── 6
4 ──┤     ├── 5
```

So in the breadboard at columns 35–38 straddling the gap: - Pin 1 (output A) → column 35, row e - Pin 2 (−in A) → column 36, row e - Pin 3 (+in A) → column 37, row e - Pin 4 (V−) → column 38, row e - Pin 5 (+in B) → column 38, row f - Pin 6 (−in B) → column 37, row f - Pin 7 (out B) → column 36, row f - Pin 8 (V+) → column 35, row f

3. Wire pin 4 (V−) to GND. Jumper from `38e` to top-blue rail.

4. Wire pin 8 (V+) to +5V. Jumper from `35f` to top-red rail. The IC body sits across rows e–f at the center gap; route the wire down through rows g, h, i to row j (the outermost row on the f-side of the board), then sideways to the top-red rail.

> **BB830 row reminder:** Each terminal strip on the BB830 has 10 rows of holes — `a` through `e` above the center gap, `f` through `j` below it. Row `j` is the outermost on the f-side. The center gap is what splits the IC's two rows of pins.

5. Place a 0.1 µF decoupling cap *between* pin 8 and the nearest GND rail. As close to the IC as possible — within 1 cm. One leg in row j next to where you came off pin 8, the other leg into the bottom-blue (GND) rail.

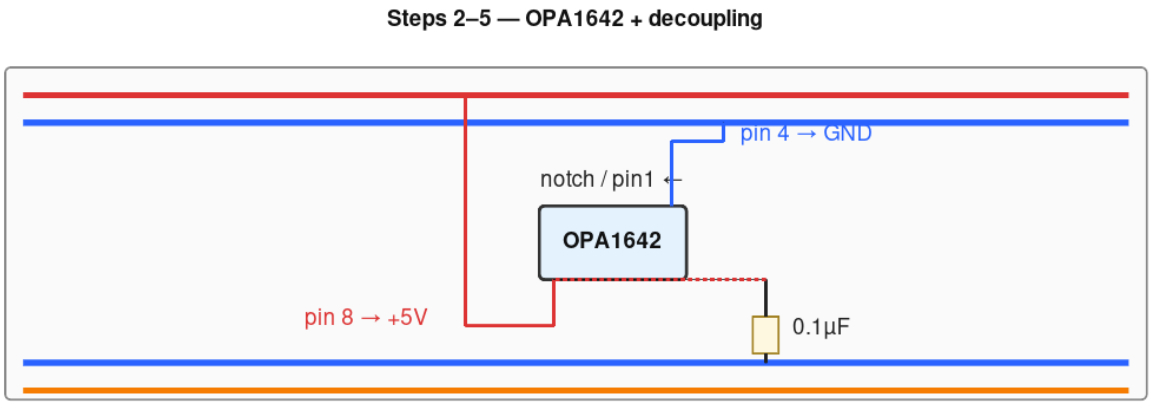


Figure 16: Phase 4 steps 2–5 — OPA1642 + decoupling

6. Build the input bias path. This is what holds pin 3 (+in) at VGND so the op-amp has a defined operating point. Two pieces: - Place a **100kΩ resistor** lying horizontally with one leg at `37a` (the pin-3 column on the a–e side) and the other leg at `45a` (a free column 8 spots to the left). The two legs must be in *different* column strips so the resistor actually carries current — same-column legs would short the resistor across one node and do nothing. - Add a **jumper from `45a` down to the bottom-red rail (VGND)**.

This puts the resistor between pin 3 and VGND, which biases pin 3 at the midpoint of the supply.

7. Place the input AC-coupling cap C1 (10 nF ceramic). One leg at `30a`, the other at `33a`. Then jumper from `33a` to `37a` (so the cap sits between the test-signal input and pin 3 of the op-amp).

8. The signal-input pad will be column 30. This is where the 3.5 mm test cable's tip wire will go.

9. Build the unity-gain feedback. Pin 1 (output A) needs to connect to pin 2 (−in A). Use a *very short* jumper from `35e` to `36e` — these are adjacent columns on the same side of the IC.

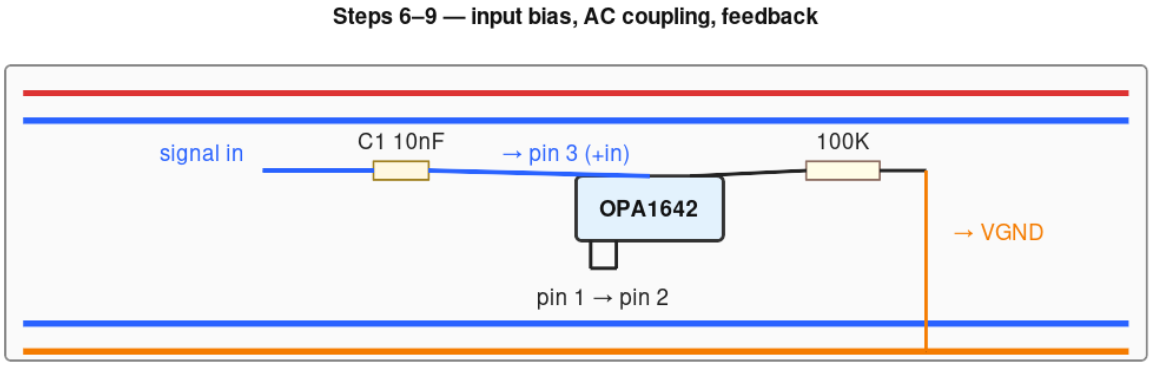


Figure 17: Phase 4 steps 6–9 — input bias, AC coupling, feedback

10. Terminate the unused half of the op-amp so it doesn't oscillate: - **Pin 5 (+in B)** → VGND rail. Jumper from `38f` down to the bottom-red rail. - **Pin 6 (−in B)** to **pin 7 (out B)** with a short jumper from `37f` to `36f`.

11. Place the output AC-coupling cap (10 µF electrolytic). Long leg (+) in `35e` (shares the strip with op-amp pin 1). Short leg (−) in a free column further out, say `48e`.

12. Run a wire from 48e to the Audio Shield's LINE-IN-L pad. Use a **3.5 mm-to-bare-wires breakout cable**: plug it into the Audio Shield's stereo line-in jack, connect the tip (left) wire to 48e, and the sleeve (ground) wire to the GND rail. This is reversible, can't damage the codec, and is the right choice for breadboard work.

> **Alternative (only if you've done codec-pad soldering before):** Solder a short pigtail directly to the LINE-IN-L silkscreen pad on the Audio Shield Rev D. The pads are 0.1″ through-holes — easier than the OPA1642 you just survived — but the breakout cable is still the right primary path because nothing you do on the breadboard can stress it.

13. Prepare the test source. Cut a 3.5 mm stereo plug or use a breakout cable to expose tip / ring / sleeve. - **Tip** (left audio) → breadboard column 30 (the C1 input side). - **Sleeve** (ground) → top-blue GND rail. - **Ring** (right audio) — not used. Tape it off.

End-of-phase verify

First — DMM checks before any signal:

[STOP] Check	DMM probes	Pass
Pin 8 reads +5V	red on the +5V rail near the IC, black on GND	4.85–5.10V
Pin 4 reads 0V	red on GND-side wire from pin 4, black on GND	< 0.05V
Pin 3 (+in) is biased to VGND	red on 37a, black on GND	2.45–2.55V
Pin 1 (output) tracks pin 3	red on 35e, black on GND	2.45–2.55V (unity-gain follower)
C2 output is DC-free	red on 48e, black on GND	< 50 mV (the cap blocks the 2.5V DC)

Then — with a signal:

Plug a phone into the test cable. Play a song at low volume. Open the Serial monitor. The Teensy's live spectrum should now show **SOURCE_NODE** energized (broadband response moving with the music). The RETURN_NODE column stays silent. Peak-level reading on SOURCE should sit in the −24 to −6 dBFS range with the phone at moderate volume.

Background

Why a JFET op-amp? Guitar pickups are high-impedance (6–250 kΩ). A bipolar-input op-amp has input bias currents in the microamp range, which over time would charge the input coupling cap and drift the operating point. JFET inputs sit at single-digit picoamps — essentially no leakage, the bias point holds indefinitely.

Why AC coupling on both ends? The op-amp output sits at 2.5V DC (the VGND reference). The codec line-in expects signals centered on its own internal bias, which is *not* 2.5V. The output coupling cap blocks DC mismatch while passing AC. Same on the input — your phone outputs AC centered on its own ground, which has no idea what our VGND is. Cap blocks DC, passes signal.

Input AC-coupling corner frequency. With C1 = 10 nF and the bias resistor (the OPA1642's input plus our 100K bias to VGND), the high-pass corner is roughly $1 / (2\pi \cdot R \cdot C) \approx 1 / (2\pi \cdot 100\,000 \cdot 10^{-8}) \approx$ **160 Hz**. That's *high* for guitar (you'll lose the low E string fundamental at 82 Hz). For a real bench measurement you'd want **C1 = 100 nF or 1 µF** for a corner near 16 Hz or

1.6 Hz. For breadboard validation of the routing topology, 160 Hz is fine — you'll still see clear signal above that.

If you want full bass response on the breadboard, swap C1 for **1× 0.1µF ceramic** (you have 20 of them) — that gives ~16 Hz corner, well below the guitar range.

Unused channel termination. The OPA1642 is a dual op-amp — two independent op-amps in one package. We're only using channel A. If channel B's inputs are left floating, it can oscillate at hundreds of MHz, which radiates back through the power pins and shows up as noise in channel A. Tying +in B to VGND and shorting −in B to out B configures channel B as a stable unity-gain follower of its own bias — the cleanest "do nothing" state.

Phase 5 — First real instrument jack (Switchcraft #11)

> **Goal:** Replace the temporary 3.5 mm test cable with a real 1/4" Switchcraft #11 jack on a flying-lead pigtail, and integrate the jack-detect switch contact with the firmware. Because the Switchcraft #11's switch is **shorted-to-tip when empty, open when inserted**, you'll invert the logic with a one-line firmware define.

Parts tray

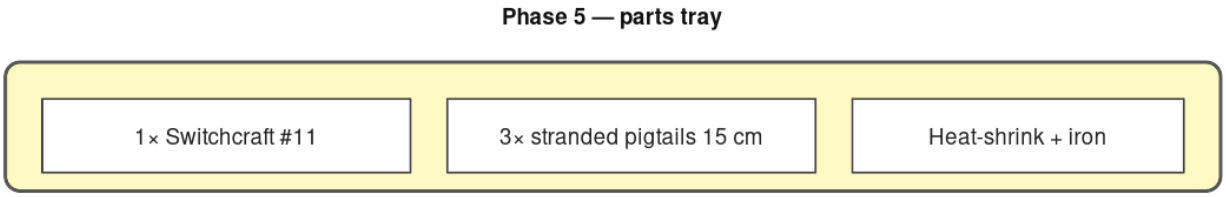


Figure 18: Phase 5 — parts tray

Switchcraft #11 terminals

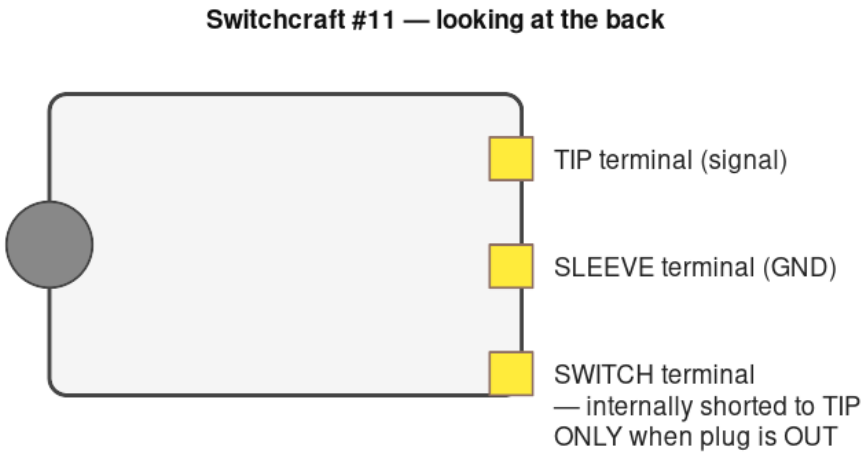


Figure 19: Switchcraft #11 terminals — looking at the back

So the switch terminal's behavior is the **opposite** of what the firmware expects:

State	Switchcraft #11 SWITCH terminal	Firmware wants
No plug	shorted to TIP terminal	HIGH (not inserted) ✓ matches if we wire TIP to GND but TIP carries signal… so it doesn't ✗
Plug inserted	open (floats)	LOW (inserted) — needs INPUT_PULLUP to pull *down* but it pulls *up* ✗

The cleanest fix is a **one-line firmware invert.** Add a `JACK_DETECT_INVERTED` define in `config.h` and XOR the read in `signal_router.cpp`.

Steps

Solder the pigtail

1. Cut three 15 cm pieces of stranded wire. Different colors are helpful: red = tip, black = sleeve, green = switch.

2. Strip 5 mm at one end of each wire and tin it. Brief touch with the iron + a thin coat of solder.

3. Slide a 1 cm piece of heat-shrink onto each wire before soldering.

4. Solder each wire to the corresponding Switchcraft #11 lug. Hold the wire in tweezers, touch the iron to the lug + wire junction, feed solder. Slide the heat-shrink down and shrink it with the iron's barrel from a short distance.

Wire into the breadboard

5. Unplug USB.

6. Remove the temporary 3.5 mm cable from phase 4.

7. Strip and tin the breadboard end of each pigtail.

8. TIP wire → breadboard column 30 (where the test cable's tip was). This is the signal input to the C1 cap from phase 4.

9. SLEEVE wire → top-blue GND rail.

10. SWITCH wire → Teensy pin 16.

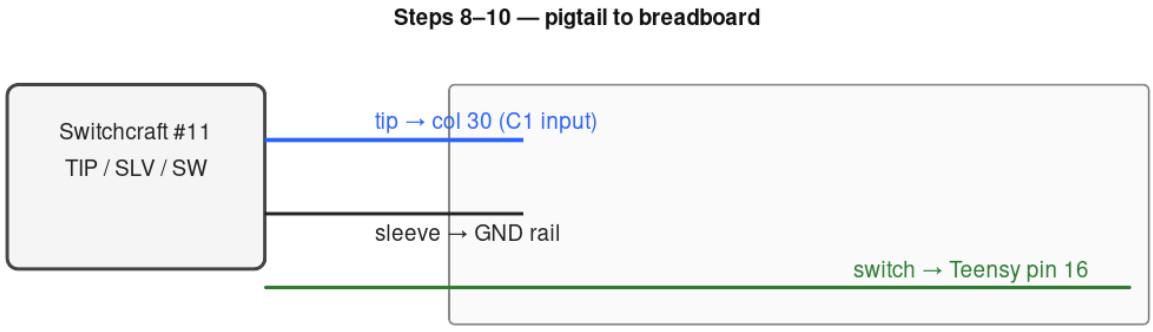


Figure 20: Phase 5 steps 8–10 — pigtail to breadboard

Firmware invert

11. Open `config.h`. Add this line in the SIGNAL ROUTER section (right after `JACK_RETURN_XLR_PIN`):

```
// Switchcraft #11 jacks have an inverted switch sense.
// 1 = pin reads HIGH when jack inserted; 0 = pin reads LOW when jack inserted.
#define JACK_DETECT_INVERTED   1
```

12. Open `signal_router.cpp`. Find the `readJackInserted()` helper (around line 70):

```
static inline bool readJackInserted(uint8_t pin) {
    return (digitalRead(pin) == LOW);
}
```

Change it to:

```
static inline bool readJackInserted(uint8_t pin) {
#if JACK_DETECT_INVERTED
    return (digitalRead(pin) == HIGH);
#else
    return (digitalRead(pin) == LOW);
#endif
}
```

13. Build + flash + open Serial monitor.

End-of-phase verify

[STOP] Check	Expected
Boot banner shows `instrument=0` when jack is empty	Yes
Insert a dummy 1/4” plug (not connected to anything)	`JACK_CHANGED  (no return, no injection)` and banner now shows `instrument=1`
Pull the dummy plug out	`JACK_CHANGED` event again, back to `instrument=0`
Plug a guitar in, strum a chord	SOURCE spectrum lights up on the display; peak meter follows the strum
Pull the plug — SOURCE spectrum collapses	Yes

Background

Why the invert? Switchcraft #11 was originally designed for “battery disconnect when unplugged” use — like a stomp-box. With no plug, the switch terminal shorts to the tip terminal (which is also connected to the battery’s negative through the circuit), pulling the battery’s ground to the chassis ground via the cable’s sleeve. Plug in → switch opens → battery (-) disconnected from chassis → circuit powered. That topology is great for stomp boxes but mismatches our use, where we want “switch closed when plug present.”

There’s an alternative — buy **Cliff CL1384** (make-on-insertion) instead — but you already have 10 Switchcraft #11s, so the one-line firmware change is cheaper. The `JACK_DETECT_INVERTED` define lets a future PCB revision use either jack type without code duplication.

Phase 6 — First relay driver (RELAY_SEND)

Goal: Build the transistor driver + flyback diode + relay coil for RELAY_SEND, energised by Teensy pin 14 going HIGH. Verify the contacts switch with a continuity test. Don't wire signal through the contacts yet — that's phase 7.

Wait for the TQ2-5V to arrive before doing this phase. TQ2-L2-5V (latching) won't work with the current firmware.

Parts tray

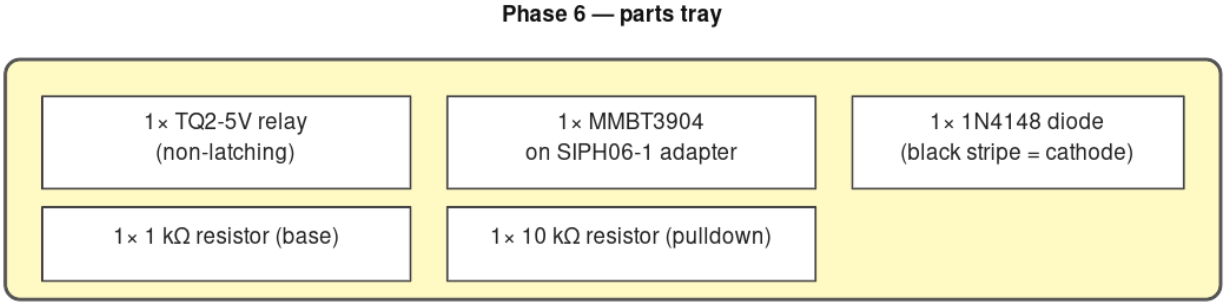


Figure 21: Phase 6 — parts tray

Resistor substitutions vs v1. v1 specced 4.7 kΩ base + 10 kΩ pulldown. With Teensy's 3.3V GPIO, a 1 kΩ base resistor gives the BJT ~2.6 mA of base drive — plenty to saturate it for the TQ2-5V's ~28 mA coil (Panasonic datasheet: 178 Ω coil at 5 V, 140 mW nominal). Stick with 1 kΩ (which you have).

Steps

1. Unplug USB.

2. Place the MMBT3904 adapter at column 50, straddling the center gap. The SIPH06-1 carries the SOT-23 to a 3-pin SIL header. Orient the adapter so its silkscreen reads **E – B – C** left-to-right (E nearest column 49, C nearest column 51) and push all three pins firmly into row e (a–e half). This means: - Emitter (E) → `49e` - Base (B) → `50e` - Collector (C) → `51e` - *Sanity-check with the DMM in continuity mode: probe each header pin back to the SOT-23 pad and confirm it matches the label — you tested this in Phase 0.5.*

3. 1 kΩ base resistor: one leg in `50a` (same strip as Base, since column 50 row a–e is one node), the other leg in `45a` (a free column to the left).

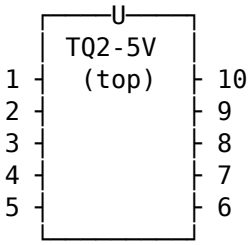
4. Jumper from `45a` → Teensy pin 14.

5. 10 kΩ base pulldown: one leg in `50b` (still on the Base strip), the other leg into the top-blue GND rail. This holds the BJT off when the GPIO is floating (during reset/boot).

6. Emitter to GND: jumper from `49a` (Emitter strip) to top-blue GND.

7. Place the TQ2-5V relay at columns 55–59, straddling the center gap. Pin 1 (marked with a small triangle, circle, or "1" on the case) goes to `55a`. The TQ2-5V is a 10-pin DIP-form package; pins march down the left side (a–e half) from `55a` through `59a`, then continue up the right side (f–j half) from `59f` through `55f`. Pinout (looking at the top, pin-1 indicator at upper-left), verified against the **Panasonic ASCTB14E datasheet, 2014-07**:

```
pin-1 indicator
 ↓
```



Pin	Function	Breadboard hole
1	**COIL (−)**	`55a`
2	**POLE 1 NC**	`56a`
3	**POLE 1 COM**	`57a`
4	**POLE 1 NO**	`58a`
5	(mechanical only — no electrical connection)	`59a`
6	(mechanical only — no electrical connection)	`59f`
7	POLE 2 NO (not used in Phase 6)	`58f`
8	POLE 2 COM (not used in Phase 6)	`57f`
9	POLE 2 NC (not used in Phase 6)	`56f`
10	**COIL (+)**	`55f`

We use POLE 1 only in this phase. POLE 2 is left unconnected.

8. Wire COIL (−) → collector of the MMBT3904. Jumper from `55a` (pin 1) to `51b` (Collector strip — same node as `51a`).

9. Wire COIL (+) → +5V rail. Jumper from `55f` (pin 10) to the top-red +5V rail.

10. Place the 1N4148 flyback diode across the coil, with the **cathode band (the black/silver stripe) toward COIL (+) — pin 10, the +5V side.** One leg in `55a` (pin 1, COIL −), the other (banded) leg in `55f` (pin 10, COIL +). In normal operation the diode is reverse-biased and does nothing; when the GPIO drops low and the coil's magnetic field collapses, the diode forward-biases and dumps the kickback voltage harmlessly into the +5V rail. *Reversed: the diode shorts +5V to GND through the BJT whenever pin 14 goes HIGH. Get this right — a multimeter in diode mode reads ~0.65 V from band-side to non-band-side and OL the other way.*

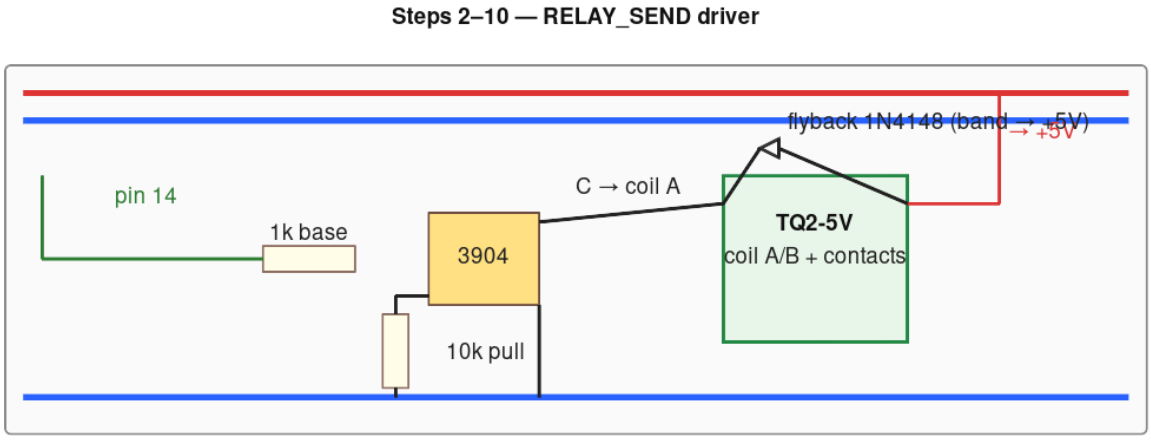


Figure 22: Phase 6 steps 2–10 — RELAY_SEND driver

Firmware: temporary toggle command

11. Open `guitar_rig_analyzer.ino`. Find the Serial command switch (around line 438). Add a temporary case:

```
// PHASE 6 BRING-UP — temporary command, remove before phase 7
case 'p': case 'P': {
    static bool s_send_relay = false;
    s_send_relay = !s_send_relay;
    digitalWrite(RELAY_SEND_PIN, s_send_relay ? HIGH : LOW);
    Serial.printf("[BringUp] RELAY_SEND -> %s\n",
                  s_send_relay ? "HIGH (energised)" : "LOW (de-energised)");
    break;
}
```

12. Build + flash + Serial monitor.

End-of-phase verify

Type `p` and `P` alternately at the Serial monitor.

Action	DMM probes	Expected
Type `p` (energise)	red on collector (coil A column), black on GND	0.1–0.3 V (saturated BJT)
Same	red on coil B column, black on GND	~5 V
Same	continuity, COM_1 ↔ NO_1 contacts	Beep (relay clicked closed)
Same	continuity, COM_1 ↔ NC_1 contacts	Open
Type `P` (de-energise)	continuity, COM_1 ↔ NC_1	Beep (relay clicked back)
Same	continuity, COM_1 ↔ NO_1	Open
Listen for a soft “click” each time		Yes

13. Remove the `p`/`P` Serial command before phase 7.

Background

Why a transistor instead of driving the coil directly? Teensy 4.1 GPIOs are rated for ~10 mA continuous, ~25 mA burst. The TQ2-5V coil draws ~28 mA nominal (5 V / 178 Ω from the Panasonic datasheet; worst-case ~31 mA with coil-resistance tolerance and temperature) — over the continuous limit and close enough to the burst limit that you cannot drive it directly without degrading the GPIO over time. A small NPN BJT (or N-MOSFET) configured as a switch lets the GPIO control a much larger load with negligible GPIO current.

Why the flyback diode? Relay coils are inductors (a coil of wire wrapped around a soft-iron core). When you suddenly cut current through an inductor (BJT switches off), the magnetic field collapses and induces a *reverse-polarity voltage spike* — easily 50–100 V on a small relay. That spike will fry the BJT’s collector-emitter junction or the BJT’s substrate diode. The 1N4148 across the coil clamps the spike to ~+5V plus 0.7V — well below the BJT’s 40 V rating.

Diode polarity. The cathode (banded end) goes to the *positive* coil terminal. In normal operation the diode is reverse-biased (looks like an open circuit). When the BJT switches off and the coil voltage flips, the diode becomes forward-biased and shunts the energy back into the +5V rail (the supply absorbs the spike).

Pulldown resistor R4. During a Teensy reset, GPIOs briefly become inputs (high-impedance). Without R4, the BJT's base would float to whatever voltage was last on the trace — possibly enough to partially turn on the BJT and chatter the relay. The 10 kΩ to GND keeps the BJT firmly off during this window.

Phase 7 — RELAY_RETURN + signal wiring + Rig Send + Rig Return TS

> **Goal:** Duplicate the phase-6 driver for the second relay (pin 15), then wire both relays into the actual signal path. Add the Rig Send and Rig Return TS jacks with their detect contacts.

This is the biggest phase by part count. Take it slow.

Parts tray

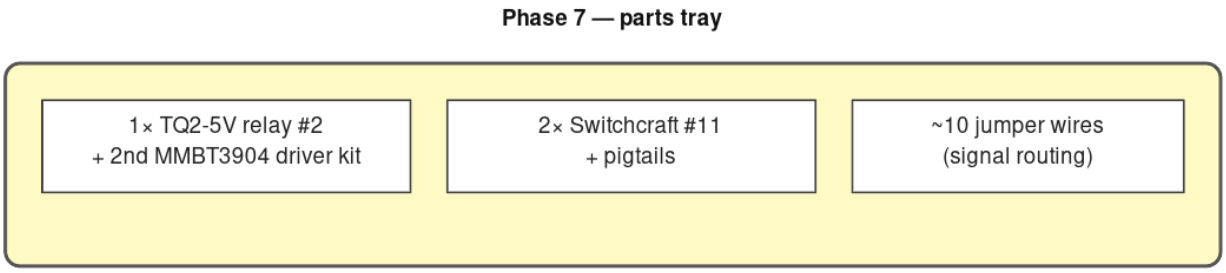


Figure 23: Phase 7 — parts tray

Steps

Duplicate the driver for RELAY_RETURN

1. Build a mirror of phase 6 at columns 60–70, but driven from Teensy **pin 15** instead of 14. All components: MMBT3904 on adapter, 1 kΩ base, 10 kΩ pulldown, TQ2-5V #2, 1N4148 flyback. Verify with the same DMM checks as phase 6 (use a quick temporary `q`/`Q` Serial command to toggle pin 15 if you want — or just lean on `signalRouter_setReturnPreference(XLR)` later to drive it).

Wire RELAY_SEND into the signal path

> **v3 critical fix.** In v2, step 2 told you to disconnect the buffer→Line-In-L wire and step 3 wired the buffer to RELAY_SEND COM with the Rig Send tip on NC and the codec line-out on NO. That topology has two bugs: (a) it severs the permanent SOURCE_NODE tap that the contract requires for self-monitoring, and (b) when injection mode energises the relay, COM↔NO connects the buffer output directly to the codec's line-out driver (two outputs fighting on one node) while Rig Send floats. v3 keeps the SOURCE tap and re-wires the relay so **Rig Send is on COM** — the standard SPDT-selector pattern.

2. Leave the existing wire from buffer output (48e) to Audio Shield Line-In L *in place*. This is the permanent SOURCE_NODE tap from Phase 4 — the contract requires it for self-monitoring. Do NOT disconnect it.

3. Add a second tap (parallel wire) from buffer output (48e) → RELAY_SEND NC_1 contact. The buffer output is now branching to two places: a permanent tap to Line-In L, and a switched feed into the relay.

4. Wire RELAY_SEND COM_1 contact → the wire that goes to the Rig Send 1/4" jack tip. (You'll add the Rig Send jack in step 10.) COM is the **switched output** that ends up at the rig.

5. Wire RELAY_SEND NO_1 contact → Audio Shield Line-Out L pad. This is the codec generator output. It's only connected to Rig Send when the relay is energised (injection mode).

Relay-state truth table (RELAY_SEND, pole 1):

State	Closed contact	Signal path
De-energised (LOW, normal play)	COM↔NC	buffer → Rig Send (the rig under test)
Energised (HIGH, self-cal)	COM↔NO	codec Line-Out → Rig Send (generator injection)

Either way, the permanent buffer→Line-In L tap from step 2 keeps SOURCE_NODE alive — that's how the firmware sees what's being sent into the rig regardless of which signal is driving it.

Wire RELAY_RETURN into the signal path

The return path uses the same SPDT-selector pattern: COM is the destination (codec input), NC is the default source (TS jack), NO is the alternate source (XLR jack from Phase 8).

6. Wire RELAY_RETURN NC_1 contact → wire that will go to Rig Return TS jack tip. When RELAY_RETURN is **de-energised (LOW)**, COM↔NC closes — the TS jack feeds the codec.

7. Wire RELAY_RETURN NO_1 contact → wire that will go to XLR pin 2. *(Leave the wire un-terminated for now; the XLR comes in phase 8.)* When energised, COM↔NO closes — the XLR feeds the codec.

8. Wire RELAY_RETURN COM_1 contact → Audio Shield Line-In R pad. This is the RETURN_NODE — the post-rig measurement point. The codec always sees whichever return source the relay has selected.

Add the Rig Send + Rig Return TS jacks

9. Solder pigtails to Jack #2 (Rig Send) and Jack #3 (Rig Return TS). Same technique as phase 5.

10. Jack #2 (Rig Send) wiring: - TIP wire → RELAY_SEND **COM_1** contact (the wire you left dangling in step 4). - SLEEVE wire → GND rail. - SWITCH wire → Teensy pin 17 (JACK_RIG_SEND_PIN).

11. Jack #3 (Rig Return TS) wiring: - TIP wire → RELAY_RETURN NC_1 contact (from step 6). - SLEEVE wire → GND rail. - SWITCH wire → Teensy pin 22 (JACK_RETURN_TS_PIN).

End-of-phase verify

[STOP] Action	Expected
Plug a guitar into Instrument In, send → rig, rig → return TS	Live SOURCE + RETURN both light up; transfer-function curve visible
Unplug Rig Return TS	`JACK_CHANGED + ROUTE_INVALID`

[STOP] Action	Expected
Arm a self-cal capture (Serial `k`)	RELAY_SEND audibly clicks during the capture (NO contact engages, codec line-out drives Rig Send)
At idle, RELAY_SEND COM↔NC closed	Continuity beep
At idle, RELAY_SEND COM↔NO open	No beep

[STOP] Do not move on until you can pull each plug and see the correct route-state transition.

Background

Signal routing logic in firmware. With both relays de-energised (default), the path is: - Instrument In → buffer → RELAY_SEND NC → Rig Send → [rig] → Rig Return TS → RELAY_RETURN NC → codec line-in R.

When the firmware needs to inject the internal generator (for self-calibration or internal-stimulus measurements), `signalRouter_setInjectionEnabled(true)` energises RELAY_SEND, switching its COM from the buffer to the codec line-out. The generator's audio leaves the codec, hits the relay, goes to Rig Send, through the rig, back to Rig Return, and into the codec line-in R.

When a return XLR is preferred over TS, `applyReturnRelay(XLR)` energises RELAY_RETURN, switching its COM from the TS jack's tip to the XLR's pin 2.

The current firmware in `signal_router.cpp` does both of these correctly; you've just connected the wires it expects to drive.

Phase 8 — XLR return

> **Goal:** Add the Neutrik XLR jack as a second return source. Standard XLRs don't have a switch contact, so we use a small mechanical microswitch actuated by the inserted XLR shell.

Parts tray

> **Note about NC3FAH2-0:** Neutrik's NC3FAH2-0 is a **combo XLR/TRS jack** — it accepts both XLR plugs *and* 1/4" TRS plugs in the same hole. Pin layout for the XLR portion: pin 1 = GND, pin 2 = signal (+), pin 3 = signal (−). The TRS portion shares the same pins via internal switching. For our use, treat it as XLR-only — wire pin 2 as the signal hot, leave pin 3 going to GND for now (single-ended use, see note below).

Steps

1. Solder pigtails to the XLR jack: - **Black wire → pin 1 (GND)** - **Red or yellow wire → pin 2 (signal hot)** - **Green or blue wire → pin 3 (signal cold — wire to GND in this single-ended build)**

2. Mount the XLR jack next to the TS jacks on whatever you're using as a panel (cardboard, plate, enclosure).

3. Mount the D2F-01L microswitch next to the XLR jack so the lever is *pressed by the XLR plug shell as it goes in*. Use double-sided tape, hot glue, or a tiny bracket. Test mechanically

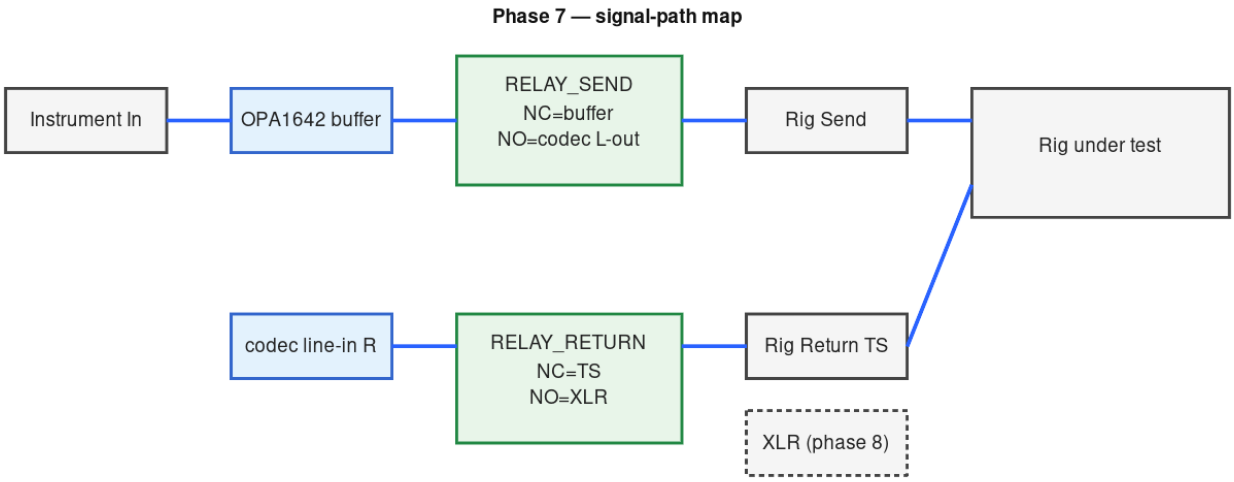


Figure 24: Phase 7 — signal-path map (v3 corrected topology)

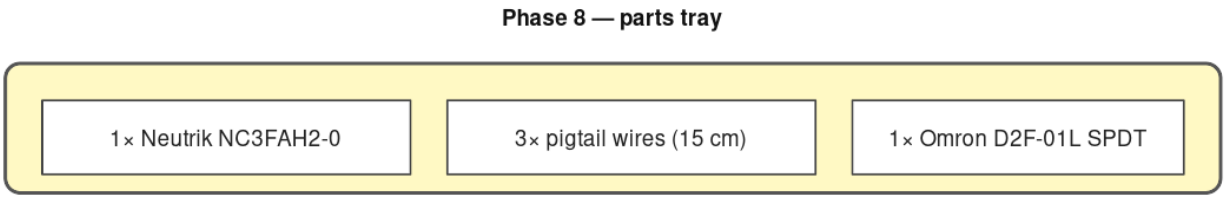


Figure 25: Phase 8 — parts tray

before wiring: insert a dummy XLR plug and confirm you can hear/feel the switch click as the lever depresses.

4. Wire the D2F-01L's three PCB terminals. Looking at the switch with the lever side up, the three pins are **COM** (the middle pin, usually marked), **NC** (normally-closed), and **NO** (normally-open). The wiring:

- **COM** → Teensy **pin 24** (JACK_RETURN_XLR_PIN).
- **NC** → **GND** rail.
- **NO** → leave **unconnected**.

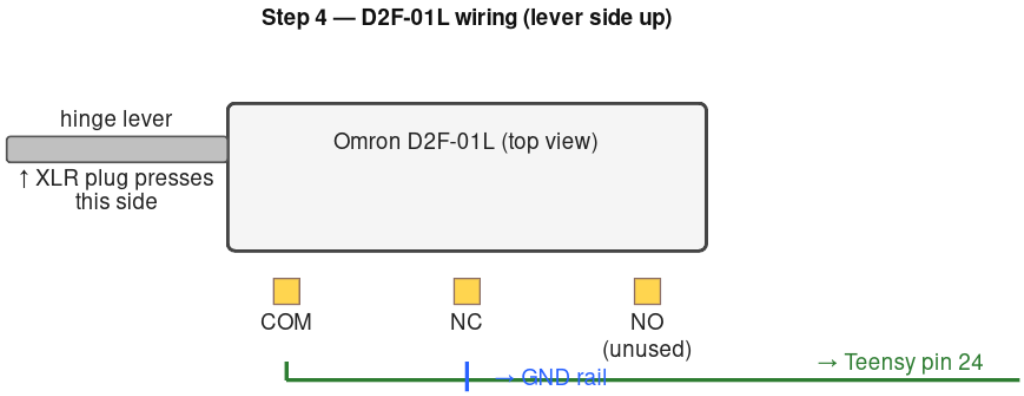


Figure 26: Phase 8 — microswitch wiring (NC default = empty)

> **Why this wiring works with the inverted firmware logic from Phase 5.** With no plug, the lever is at rest → COM-NC is closed → pin 24 is pulled to GND → reads LOW → firmware (with `JACK_DETECT_INVERTED=1`) interprets that as "**not inserted.**" When the XLR plug goes in and presses the lever, COM-NC opens → INPUT_PULLUP floats pin 24 → reads HIGH → firmware interprets that as "**inserted.**" Both the Switchcraft #11s and this microswitch end up speaking the same sense convention.

5. Wire XLR pin 1 → GND rail.

6. Wire XLR pin 2 → RELAY_RETURN NO_1 contact (the wire you left dangling at the end of phase 7 step 7).

7. Wire XLR pin 3 → GND rail (single-ended use for breadboard only — the PCB will use a proper instrumentation-amp differential input stage).

End-of-phase verify

[STOP] Check	Expected
Plug an XLR cable into the jack; the microswitch's lever depresses	`JACK_CHANGED` event, `ROUTE_CHANGED → XLR return` (assuming TS is also unplugged, or XLR is preferred via `signalRouter_setReturnPreference(XLR)`)
Hum into a dynamic mic plugged into the XLR	Live spectrum shows return signal on RETURN_NODE
Unplug the XLR	Microswitch lever springs back; route flips back to NONE or TS

Background

Why a microswitch instead of a jack-integrated detect? Most cheap panel-mount XLRs (including the Neutrik NC3FAH2-0) don't have a built-in switch contact. The professional move is to use a **Neutrik NC3FAV-SW** (a variant with switch contacts) on the PCB, but for the breadboard the microswitch is cheap and works fine.

Why pin 3 to GND instead of to the inverting input of a differential amp? A balanced XLR signal carries the audio as `pin 2 − pin 3`. To recover it correctly you need an *instrumentation amplifier* — the standard chip is the **INA1620** (designed specifically for balanced audio). For the breadboard topology proof you can get away with treating XLR as single-ended (pin 2 = signal, pin 3 = ground), losing the common-mode noise rejection that balanced audio is famous for. This is **not** acceptable for the production PCB — note it in your build log so it doesn't sneak into the final design.

Phase 9 — Validation harness

> **Goal:** Use the firmware's built-in 4-test validation suite to qualify the breadboard. Passing this is the sign-off for going to PCB.

Parts tray

Just the firmware's existing `validation_runAll()`. Optional helpers: - A Y-split 1/4" TS cable (or two 1/4" plugs + a short wire) — for Test 1. - A simple resistive pad (2× equal resistors, e.g., 10 kΩ series + 10 kΩ shunt = 6 dB) — for Test 3. - An RC filter (e.g., 10 kΩ + 10 nF — you have both — = 1.6 kHz corner) — for Test 4.

Steps

1. **Open Serial monitor at 115200 baud.**
2. **Run validation tests one at a time.** Type each key in sequence:

Key	Test	Setup	Pass criterion
1	Flat loopback	Y-split so Instrument In + Rig Send are joined, and Rig Return TS receives the same node	`tf_db[]` flat within tolerance, 200 Hz – 10 kHz
2	Channel-swap symmetry	Same physical Y-split; firmware re-binds channels mid-test	Asymmetry within tolerance
3	Known gain offset	Insert a 6 dB resistive pad on the return path	Measured offset = expected ± tolerance
4	Known RC filter	Insert RC low-pass on the return path	Slope + `tf_at_2fc` match expected

3. **Or run them all at once with `v`** — the firmware does a preflight level check first, then steps through all four.

4. Capture the Serial output to a text file (Arduino IDE's serial monitor has a "save" button, or you can `pbpaste` it).

> **Heads-up on Test 1's low end.** Phase 4 uses a 10 nF input AC-coupling cap, which combined with the 100 kΩ bias resistor gives an input high-pass corner at ~160 Hz. Test 1's pass band is 200 Hz – 10 kHz, so it'll pass, but if you peek at the raw `tf_db[]` below 300 Hz you'll see extra rolloff. That's the cap, **not** a fault. To get a flat response down to 16 Hz on the breadboard, swap C1 from 10 nF to 0.1 µF (you have 20 of these in the bin).

Sign-off checklist

Each check is one line you read out loud. Tick each as you confirm.

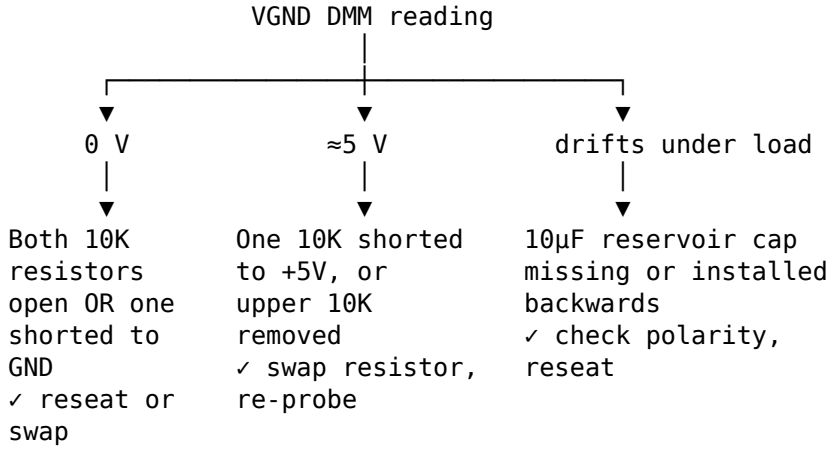
- ☐ Phase 1: VGND stays at 2.4 – 2.6 V even when relays are clicking.
- ☐ Phase 2: LED smoke test toggles cleanly at boot.
- ☐ Phase 3: All four GPIOs respond to GND taps.
- ☐ Phase 4: Phone test signal visible on SOURCE spectrum.
- ☐ Phase 5: Switchcraft #11 + invert define both work — guitar plays through, detect events fire.
- ☐ Phase 6: TQ2-5V #1 clicks on `p`/`P`; continuity matches expected state.
- ☐ Phase 7: Full route matrix (4 jack combinations) produces correct route description on Serial.
- ☐ Phase 8: XLR + microswitch detect both work end-to-end.
- ☐ Phase 9: `1`, `2`, `3`, `4` all PASS.

When every box is ticked: photograph the entire board, save the serial logs, and update docs/bench_runbook.md §8 with the date. The breadboard is signed off and ready to translate into a KiCad schematic.

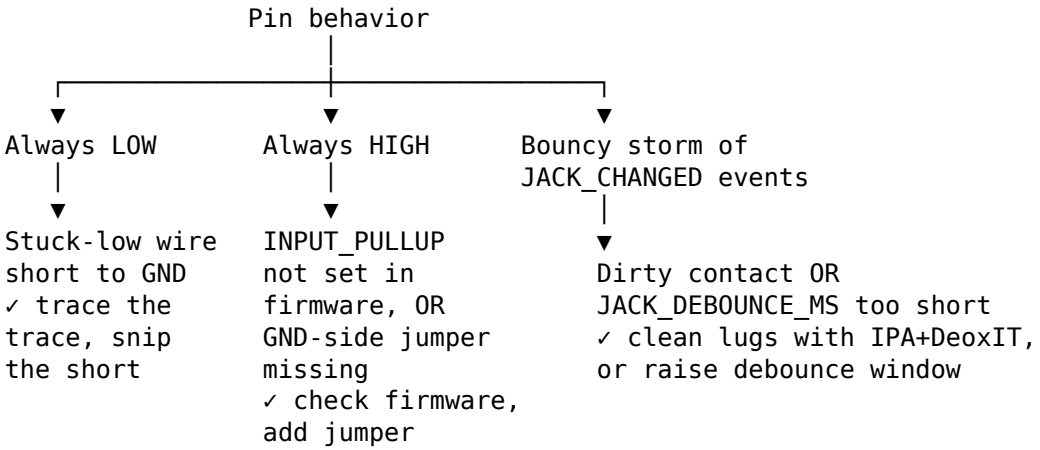
Appendix A — Troubleshooting decision tree

For every symptom below, follow the arrows until you get to a green ✓ (fixable on the bench) or a red ✗ (probably a dead part — swap from your spares bin and retest).

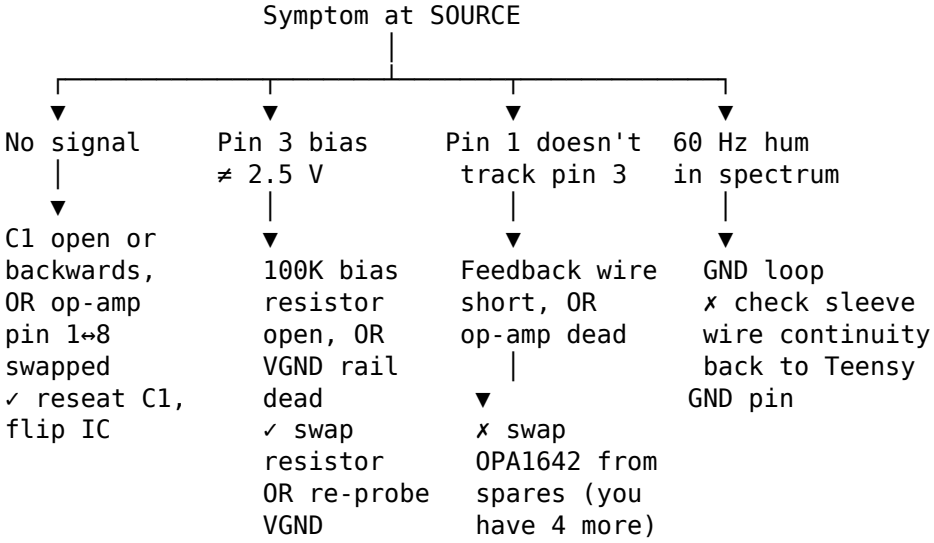
Power phase



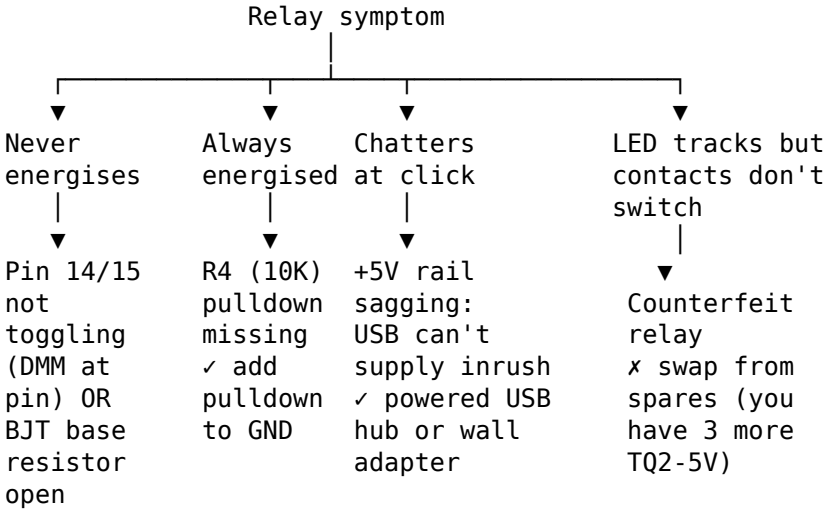
GPIO phase (smoke test, jack-detect)



Buffer phase (Phase 4)



Relay-driver phase (Phase 6, Phase 7)



```
✓ reseat,
re-flash
```

Jack-detect phase (Phase 5, Phase 8)

```
            Always reads inserted when empty
                          │
                          ▼
            Inversion convention is wrong
            ✓ flip JACK_DETECT_INVERTED
            between 0 and 1 in config.h
```

Validation phase (Phase 9)

```
        Test 1 (flat loopback) fails below 300 Hz
                          │
                          ▼
                Not a fault — it's
                the 10 nF input cap
                rolling off at 160 Hz
                ✓ swap C1 to 0.1 µF
                for full guitar range
                (or accept it for the
                breadboard sanity check)
```

Appendix B — Parts substitution table

If you ever need to source replacements, here's what works:

Original (in your bin)	Substitute	Notes
OPA1642	OPA1641 (single-channel), TL072 (dual JFET)	Same pinout (TL072 is noisier but works)
MMBT3904 SOT-23	2N3904 TO-92	Drop-in if you swap the adapter for direct DIP
TQ2-5V	Omron G6K-2P-Y-DC5, G5V-2-DC5	Pinout differs — verify datasheet
1N4148 DO-35	1N914, any small-signal Si diode rated > 100 mA	Drop-in
Switchcraft #11	Switchcraft #12B, Cliff CL1384 (no firmware invert needed for CL1384)	
10K + 10K VGND divider	TLE2426 rail-splitter IC	Cleaner, quieter, ~$2

Appendix C — v3 changelog

Critical fixes (would have blocked the build in v2):

1. **Phase 7 step 2 + relay topology rewritten.** v2 disconnected the buffer→Line-In L tap (severing SOURCE_NODE per `signal_router_contract.md`) AND wired the relay with COM=buffer, NC=Rig Send, NO=codec line-out — which meant in injection mode the buffer output was shorted into the codec's line-out driver while Rig Send floated, delivering no generator signal to the rig. v3 keeps the SOURCE tap permanently in place, adds a parallel feed into the relay's NC contact, and swaps the topology so **COM=Rig Send, NC=buffer, NO=codec line-out** — the standard SPDT-selector pattern.
2. **Phase 1 TLE2426 sidebar.** v2 claimed "you didn't order a TLE2426." v3 corrects this: 5× TLE2426CLP are in stock per `parts_inventory.md:80`. The 10K/10K divider is now framed as an intentional educational choice that saves the TLE2426s for the production PCB.
3. **Phase 6 step 7 TQ2 pinout.** v2 punted to "verify from the datasheet" and left COIL_A / COM_1 / etc. as placeholders. v3 locks in the Panasonic ASCTB14E (2014-07) pin assignments: pins 1 & 10 = coil, pins 2/3/4 = POLE 1 NC/COM/NO, pins 7/8/9 = POLE 2, pins 5 & 6 are mechanical-only.
4. **Phase 4 step 6 thinking-out-loud removed.** v2 contained a "Wait — that puts both legs on the same strip and the resistor accomplishes nothing — Correction:" draft note in the published doc. v3 replaces it with the correct step in one pass.
5. **Phase 6 step 10 diode polarity rewrite.** v2 said "cathode band facing the +5V side" without naming which terminal that is. v3 says **"cathode band toward COIL (+), pin 10, the +5V side"** so it's unambiguous regardless of which reference frame the reader is in.

Major fixes (anti-Lego patterns):

6. Phase 6 step 2 SIPH06-1 row mapping committed to single hole positions (`49e` / `50e` / `51e`) instead of "row f or g".
7. Phase 4 step 4 "row j" hop given a 1-line BB830 row legend (`a–e` above the gap, `f–j` below).
8. Phase 4 step 12 promoted the breakout-cable path to primary; demoted soldered pigtail to alternative for experienced solderers.
9. Phase 5 deleted the "wiring trick to flip the sense" half-promise that v2 introduced but never described.
10. Phase 2 LED source clarified to the DIDODI practice kit (which is already in the parts inventory) instead of "any 2 LEDs lying around."

Quick wins:

11. Added a "If something feels wrong — read this first" panic card to the top of the doc (USB unplug, ESD note, spares reassurance).
12. Phase 9 now flags the 160 Hz HPF behavior so a successful test 1 below 300 Hz isn't read as a fault.
13. Appendix A rewritten as ASCII decision trees per phase, with explicit □/□ outcomes.
14. v2's auto-numbered "Figure N: diagram N" captions replaced with descriptive captions when the v3 PDF is rendered (see image alt-text in the markdown source).

Known gaps still present in v3:

- The figures are still minimal — block diagrams and parts-tray rectangles. A future v4 should replace them with dense, top-down breadboard-grid images showing every wire/component placement after each step (review's Major #5). This is a multi-day illustration effort and out of scope for v3.
- Macro photos of IC pin-1 marks, Switchcraft #11 lug labels, D2F-01L COM/NC/NO terminals, and SIPH06-1 silkscreen orientation are not included. Replace the ASCII pinout tables with annotated photos in v4.

End of v3 bring-up document. v1 is preserved at breadboard_bringup.md, v2 at breadboard_bringup_v2.md. When phase 9 signs off, update CHANGELOG.md and start the PCB layout.